

업무일지 — 이진호 (개인 프로젝트: RFP RAG 시스템)

2026-06-18 (금) · 프로젝트 이해 · 개인 전환 · 스택 결정

오늘 할 일

- 중급 프로젝트가 뭘 요구하는지 파악
- 개인 프로젝트 전환에 따른 진행 방식 확인
- 기술 스택(특히 GCP 사용 여부) 결정, 레포 셋업

한 일 / 느낀 것

- 팀용/개인용 가이드가 다르다는 걸 확인. 개인은 **클라우드 서버 미제공 → 로컬 + LLM API(시나리오 B)** 로 진행. 발표 없음, 결과물은 GitHub 레포 + 보고서 PDF + 업무일지.
- 시나리오 A(GCP+HuggingFace)와 B(API)의 차이를 정리해봤다. A는 데이터를 외부로 안 보내는 대신 GPU-서빙 부담이 크고, B는 쉽고 품질 좋지만 데이터가 외부 API로 나간다. 개인이라 **B로 완성도부터 잡고, A는 선택사항으로 남기기로.**
- Vector DB는 Chroma로 잠정 결정(로컬 영속 + 메타데이터 필터링). 임베딩/생성은 허용 모델 (`text-embedding-3-small`, `gpt-5-mini`).
- GitHub 레포 생성, 디렉토리 구조 스케치.

고민 / 실수

- API 키를 어떻게 조달할지 애매. 개인 전환이라 팀 채널 키를 못 받는 상황 → 우선 **본인 OpenAI 키**로 진행하기로. 키는 `.env` 로 분리하고 `.gitignore` 로 커밋 차단하는 습관부터 잡음.

소감

- 혼자 다 해야 해서 막막하지만, 범위를 스스로 정할 수 있는 건 오히려 자유롭다. 일단 "작게라도 돌아가는 것"부터 만들자는 마음.

2026-06-19 · 데이터 확보 · 탐색적 분석(EDA)

오늘 할 일

- 데이터 내려받고 구조 파악
- 어떤 필드가 검색/필터에 쓸모 있는지 확인

한 일 / 인사이트

- 문서 100건(hwp 96 + pdf 4)과 `data_list.csv` 메타데이터 확인.
- 메타데이터 컬럼: 공고번호, 사업명, 사업금액, 발주기관, 공개일자, 입찰마감일, 사업요약, 파일형식, 파일명, 텍스트.
- **핵심 발견:** `data_list.csv` 에 `텍스트` 컬럼이 있어 본문이 이미 어느 정도 추출돼 있음. hwp 파싱을 우선 건너뛰고 이걸로 베이스라인을 빠르게 만들 수 있겠다 판단. 단 문서당 평균 ~3,800자로 **원문 전체가 아니라 일부**로 보인다 (나중에 문제될 수 있으니 메모).
- 발주기관/사업명을 메타데이터 필터에 쓰면 다중 문서에서 타깃을 좁힐 수 있겠다는 아이디어.

고민

- 공고번호에 결측치가 꽤 있다. 문서 식별자로 쓰려면 유일성 처리가 필요하겠다 (이때는 가볍게 넘겼는데, Day 3에 이게 버그로 돌아옴).

소감

- 데이터를 열어보니 진짜 실무 RFP라 흥미롭다. 근데 한 문서가 수십 페이지라, 왜 이걸 요약·검색해주는 시스템이 필요한지 체감된다.

2026-06-22 · 베이스라인 파이프라인 구현 · 첫 버그

오늘 할 일

- 로딩→청킹→임베딩→검색→생성까지 end-to-end로 "돌아가는" 파이프라인 만들기

한 일 / 코드

- 모듈 분리해서 구현: `config.py` (하이퍼파라미터 중앙관리), `src/{data_loader, chunking, vectorstore, retriever, generator, rag}.py`, `scripts/{build_index, ask}.py`.
- 청킹은 글자수 800/중첩 150 기준으로 문단 경계를 존중하도록. 생성 프롬프트엔 **근거 기반 답변 강제 + 없는 내용은 "모른다"** 규칙을 넣어 환각을 억제.
- 검색은 우선 코사인 유사도 + MMR 옵션.

고민 / 실수 (중요)

- **DuplicateIDError:** 첫 인덱스 구축에서 터졌다. 공고번호가 빈 문서들의 `doc_id`가 전부 `nan` 이 되어 Chroma에 중복 ID로 저장 실패. Day 2에 "나중에 처리해야지" 했던 결측치가 그대로 사고로. → `doc_id`를 "공고번호 없으면 행번호, 중복이면 접미사"로 **무조건 유일하게** 만들어 해결.
- 교훈: ID 유일성은 미루지 말고 처음부터 보장할 것.

결과

- 버그 잡은 뒤 "국민연금공단 이러닝시스템 요구사항 정리" 질의에 정답 문서를 찾아 근거와 함께 답변. **베이스라인 end-to-end 성공.**

소감

- 돌아가는 걸 처음 봤을 때 짜릿했다. 근데 답이 좀 얇은 느낌... 텍스트 컬럼이 짧아서 그런가 싶다. 정량 평가를 붙여봐야 진짜 상태를 알 수 있겠다.

2026-06-23 · 평가 설계 · 평가셋/지표 정의

오늘 할 일

- "느낌"이 아니라 "숫자"로 성능을 볼 수 있게 평가 체계 만들기

한 일 / 인사이트

- 가이드의 공식 예시 질문 8개 + 기관명 부정확 입력 케이스 1개 + 없는 정보(환각 체크) 1개 = **10 케이스 평가셋**(eval/questions.json) 구성. 각 질문에 **정답 발주기관**을 라벨로 붙여 검색을 객관 채점 가능하게.
- 지표를 직접 설계: 검색 커버리지 / Hit@k / MRR / groundedness(충실도) / relevance(관련성) / abstention 정확도 / 지연시간.
- groundedness-relevance는 정답을 일일이 손으로 달 수 없으니 **LLM-as-judge(참조 없이 근거-답변 정합성 채점)**로 처리하기로.

고민

- 멀티턴(후속질문)을 평가에 어떻게 넣을지. 결국 케이스마다 턴을 순서대로 실행하고 마지막 답을 평가하는 방식으로.
- LLM 심판을 얼마나 믿을 수 있나? 완벽하진 않지만 상대 비교(설정 A vs B)엔 충분하다고 판단.

소감

- 평가셋을 만들다 보니 "이 시스템이 실제로 뭘 잘해야 하는가"가 더 선명해졌다. 단일 추출, 다중 종합, 후속 맥락, 환각 방지 — 이 네 가지구나.

2026-06-24 · 평가 하네스 완성 · 심판 버그 · 검색 약점 발견

오늘 할 일

- 평가 자동화 스크립트 완성하고 베이스라인 점수 측정

한 일 / 결과

- `scripts/evaluate.py` 완성, 베이스라인 측정:
- `retrieval_coverage` **0.5**, `hit_all` **0.375**, `mrr` **0.53**
- `abstention_accuracy` **1.0**, 평균 지연 **~11초**
- **검색이 약점이라는 게 숫자로 드러났다.** 정답 문서를 평균 절반만 검색해온다. 특히 (1) 두 기관 비교 질문은 한쪽만 걸리고, (2) 기관명을 부정확하게 쓰면 못 찾음.
- 환각 방지는 잘 됨(없는 사업엔 안 지어내고 "모른다" 100%).

고민 / 실수 (중요)

- 심판 점수가 전부 **None**으로 나와 한참 헤맸. 원인은 모델 특성이었다. `gpt-5-mini`는 **추론형 모델**이라 `max_completion_tokens`를 추론에 먼저 소모한다. 심판 호출에 250토큰만 줬더니 본문이 비어서 반환 → JSON 파싱 실패 → None. 토큰을 900으로 올리고 `response_format=json_object` 강제 + 정규식 폴백으로 해결.
- 같은 함정이 LLM 리랭커에도 있었음(토큰 100 + 배열 파싱). 조용히 실패해 기본 순서로 폴백 중이었을 것. 똑같이 고침.
- 교훈: **추론형 모델은 출력 토큰을 넉넉히, LLM 출력 파싱은 항상 실패를 가정하고 방어적으로.**

소감

- 숫자가 마음을 편하게도, 불편하게도 만든다. 잘 되는 줄 알았는데 커버리지 0.5라니. 그래도 약점을 특정했으니 방향은 분명하다.

2026-06-25 · 검색 기법 실험 비교 · 원인 분석

오늘 할 일

- 여러 검색 설정을 같은 평가셋으로 돌려 뭘 채택할지 결정

실험 비교 결과 (CSV 모드, 10케이스)

실험	coverage	mrr	groundedness	relevance	latency
1 baseline(유사도 top5)	0.562	0.542	1.43	1.43	10.9s
2 MMR top5	0.5	0.525	2.0	2.0	11.3s
3 MMR top8	0.625	0.543	2.0	2.25	9.1s
4 MMR+자동필터	0.625	0.75	1.63	1.88	7.7s
5 MMR+LLM리랭크	0.562	0.542	1.67	1.67	19.7s

분석 / 의사결정

- 자동필터가 검색 부문 최고. MRR 0.54→**0.75**, 지연도 최저(7.7s). 질의에서 기관명을 퍼지매칭해 발주기관으로 메타데이터 필터링하니 정답 문서가 1순위로 올라온다. 기초과학연구원 케이스가 cov 0→1로 살아남. → 채택.
- LLM 리랭크는 폐기. 검색 지표는 그대로인데 지연만 2배(19.7s). 후보가 이미 좋을 땐 리랭킹 이득이 없고 비용만 든다. (안 되는 것도 실험으로 근거를 남긴다.)
- top8은 coverage를 살짝 올린다 → 자동필터와 결합 가치 있음.

여기서 드러난 더 큰 두 문제

1. 답변 품질이 전반적으로 낮다(1.4~2.25/5). 검색을 잘해도 CSV 텍스트가 알아서(문서당 ~3.8천자) 세부 요구사항이 없다. → 원본 hwp/pdf 전체 파싱으로 전환 결정.
2. 후속질문이 전멸(q2, q5 cov 0.0). "콘텐츠 개발 관리 더 알려줘"엔 기관명이 없어 검색이 샌다. → 대화 맥락 기반 쿼리 재작성 도입 결정.

소감

- 실험이 다음에 뭘 해야 할지를 정확히 가리켜줬다. "안 되는 실험(리랭크)"도 근거로 남긴 게 오히려 값지다. 이제 진짜 개선 두 방을 준비한다.

2026-06-26 ~ 2026-06-30 · 원본 파싱 · 쿼리 재작성 · 개선 검증

오늘 할 일

- 원본 hwp/pdf 전체 파싱 도입, 후속질문 쿼리 재작성 추가
- 원본으로 재인덱싱해 CSV 대비 품질이 오르는지 검증

한 일 / 코드

- HWP 파서 개선: 처음 만든 파서는 본문에 제어문자 잡음(握獺 등)이 섞였다. 원인은 HWP 본문의 인라인 컨트롤 문자를 안 건너뛴 것. 컨트롤 문자가 8워드를 차지한다는 규칙을 반영해 잡음 제거. 원본 파싱 결과 문서당 평균 7.3만 자(CSV 대비 약 19배), 100/100 전부 파싱 성공.
- src/data_loader.py 에 source="raw" 옵션 + build_index raw 지원 추가.
- src/query_rewrite.py 추가: 히스토리를 반영해 후속질문을 독립 질문으로 재작성한 뒤 검색.
- 실험셋에 6(자동필터+top8), 7(자동필터+top8+쿼리재작성) 추가.

고민 / 실수 (원본 인덱싱 크래시 — 서로게이트 문제)

- build_index raw 를 돌리니 청크는 11,843개(문서당 평균 118개)로 잘 쪼개졌는데, 임베딩 3번째 배치에서 UnicodeEncodeError: surrogates not allowed 로 크래시. 인덱스가 텅 빈 채로 남았다

(build_index가 시작 시 기존 컬렉션을 지우기 때문).

- 그걸 모르고 `run_experiments` 를 돌렸더니 실험 1~7이 **전부 coverage 0.0**인데 groundedness는 4~5로 높게 나오는 이상한 결과. 처음엔 "원본이 오히려 검색을 망쳤나?" 오해했는데, 뜯어보니 **빈 인덱스라 매번 '모른다'고 답했고, LLM 심판이 그 abstention을 '환각 없음=충실도 높음'으로 보상**한 착시였다. → **교훈: groundedness가 높아도 coverage가 0이면 그건 '잘한 것'이 아니라 '아무 것도 안 한 것'이다. 지표는 항상 교차 검증해야 한다.**
- 근본 원인: HWP 원문에 BMP 밖 문자가 있는데, 내 파서가 UTF-16 단위를 글자당 하나씩 처리하면서 **서로게이트 쌍을 두 개의 외톨이 서로게이트로 쪼갬**. 이게 UTF-8/JSON으로 인코딩이 안 돼 API 전송 시 터진 것.
- 해결: (1) 파서에서 상위+하위 서로게이트 쌍을 감지해 하나의 코드포인트로 결합, (2) `_clean` 과 임베딩 직전에 `encode('utf-8', 'ignore')` 안전장치 추가. 오프라인으로 100/100 전 문서가 UTF-8 인코딩 통과하는 것 확인 후 재빌드.
- 교훈: 바이너리 포맷(HWP) 파싱은 유니코드 엡지케이스(서로게이트)를 반드시 방어해야 한다. 그리고 "성공적으로 보이는 실패"를 지표 교차검증으로 걸러내는 습관.

고민 / 실수 (Chroma 배치 상한 + 자동필터 결합 — 재실행 전 레드팀으로 선제 발견)

- 서로게이트 수정 후 재빌드하니 임베딩은 11,842개 끝까지 성공(4분). 그런데 이번엔 **Chroma add의 배치 상한(5,461)** 에 걸려 저장 실패(Batch size 11842 > max 5461). CSV 모드(616개)에선 안 걸리던 게 원본 청크가 11,842개로 커지자 터짐. → **5,000개씩 나눠서 add** 하도록 수정.
- 같은 실수를 또 반복하기 싫어서, 재실행 전에 **로컬 Chroma에 가짜 임베딩으로 11,842개 전체를 넣어보는 레드팀 테스트**를 돌렸다. 여기서 자동필터의 숨은 결합 2개를 추가로 발견:
- **비교 질문에서 자동필터가 한 기관만 잡아 나머지를 걸러냄** (고려대 vs 광주과학기술원인데 한쪽만 남음). CSV 실험서 비교질문 cov가 0.5였던 원인일 수 있다.
- 기관명 접미사(연구원/대학교/과학)의 우연한 겹침으로 **영동한 기관이 필터에 달려옴**(예: 기초과학연구원 질문에 국방과학연구소가 섞임).
- 해결: 기관명이 질의에 **60% 이상 통째로 등장할 때만** 매칭(정밀 LCS), 공백 정규화로 '한국 원자력연구원'도 대응, 여러 기관이 잡히면 `$in` 으로 모두 포함. 10개 질문 전부 올바른 필터가 나오는 것을 확인.
- 교훈: 실패를 한 번 겪었으면, 다음 실행 전에 로컬에서 돌릴 수 있는 부분은 미리 다 돌려보고 **선제적으로 잡자(레드팀)**. API가 필요 없는 Chroma·파싱·필터 로직은 전부 로컬 검증이 가능했다.

다음(진행 예정)

- 서로게이트 수정 후 `build_index raw` 재실행(성공 시 "저장 완료: 11843개 청크") → 같은 평가 재실행
- 기대: 원본 텍스트로 groundedness/relevance 상승(문제1), 실험7에서 후속질문 커버리지 회복(문제2)
- 결과 확정되면 최종 설정 픽스 + 보고서 작성

소감

- 얇은 데이터를 붙잡고 있던 게 계속 걸렸는데, 원본을 제대로 뽑아내니 속이 시원하다. 다만 원본을 넣자마자 서로게이트로 한 방 먹었다. 특히 "coverage 0인데 groundedness 5"라는 착시에 잠깐 속았던 게 뜨끔했다 — 숫자를 액면 그대로 믿으면 안 된다는 걸 몸으로 배웠다. 파서 잡음(컨트롤 문자)에 이어 서로게이트까지, 바이너리 포맷 파싱의 현실을 제대로 겪는 중.

2026-07-01 · 최종 인덱스 재빌드 · EDD 평가 · 증거 정리

오늘 할 일

- 원본(raw) 인덱스를 실제로 다시 만들고, 주요 설정별 실험 결과를 비교한다.
- 단순 평균 점수만 보지 않고 EDD score, 그래프, 질문별 원인분석으로 개선 근거를 남긴다.
- 점수가 높은 답변도 실제로 맥락에 맞는지 확인할 수 있도록 질문-답변-근거 패킷을 만든다.

한 일 / 설정

- `build_index raw` 를 재실행했다. 원본 파싱 100건, 청크 **11,842개** 저장 완료를 확인했다. 이전의 서로게이트/Chroma 배치 문제는 재발하지 않았다.
- 검색은 좋아졌지만 답변이 빈 문자열로 나오는 케이스가 있었다. q1을 직접 재현해 보니 `finish_reason=length`, `reasoning_tokens=1024`, `content=""` 였다. `gpt-5-mini` 가 1024 토큰을 전부 추론에 써버린 것이다. 그래서 생성 기본 토큰을 3072로 올리고, 빈 답변/length 종료 시 4096 이상으로 재시도하게 고쳤다.
- `top_k`, `mmr_lambda`, `fetch_k`, `auto_filter × rewrite_query` 를 조건별로 나누어 평가했다. 각 조건의 결과는 CSV/JSON으로 저장했고, 최종 판단은 같은 평가 기준으로 비교했다.
- EDD score를 정의했다. coverage, hit-all, MRR, groundedness, relevance, abstention, latency를 가중합하고, false abstention과 empty answer는 페널티로 뺐다. 결과 그래프도 별도 summary 폴더에 SVG로 남겼다.
- 보고서에서 바로 인용할 수 있도록 `report_evidence.json/md` 를 만들었다. 질문, 답변, 검색된 기관, 지표, 자동 플래그, 해석을 한 묶음으로 남겼고 총 96개 케이스가 정리됐다.

결과 (핵심 지표)

- 최상위 조건: `MMR top8 + mmr_lambda 0.5 + fetch_k 20 + 자동필터 + 쿼리재작성`
- EDD **96.57**
- coverage **1.0**, MRR **1.0**
- groundedness **4.667**, relevance **5.0**
- false abstention **0.0**, empty answer **0.0**
- top5도 EDD **96.53**으로 거의 동률이었다. 하지만 top8이 문맥 여유가 있고 기존 원본 실험 가설과 맞아서 기본값은 top8로 고정했다.
- 최종 기본 설정은 `TOP_K=8`, `FETCH_K=20`, `USE_MMR=True`, `MMR_LAMBDA=0.5`, `AUTO_FILTER=True`, `REWRITE_QUERY=True`, `RERANK=False` 로 정했다.

원인 분석

- 검색 품질은 원본 + 자동필터 + 쿼리재작성 조합에서 확실히 좋아졌다. `filter_off_rewrite_off` 는 coverage 0.625, MRR 0.417까지 떨어졌고, 특히 q1/q4/q5가 흔들렸다. 자동필터가 없으면 기관명이 질문에 있어도 raw 청크 공간에서 다른 문서가 끼어들었다.
- 자동필터와 쿼리재작성은 서로 다른 실패를 보완했다. 필터가 꺼지면 q1/q4 같은 기관명 명시 질문이 깨지고, rewrite가 꺼지면 q5 같은 후속질문이 깨졌다. 그래서 둘 중 하나만 켜는 것이 아니라 둘 다 켜야 한다.
- `mmr_lambda=0.7` 은 검색 coverage는 1.0이지만 groundedness/relevance가 3.667/4.333으로 낮아졌다. 관련성만 너무 강하게 잡으면 다양한 근거가 줄어 답변 품질이 흔들리는 것으로 보인다.
- `fetch_k=40` 은 coverage/MRR은 유지했지만 groundedness가 4.0으로 내려갔다. 후보 풀을 키우는 게 무조건 이득은 아니었다. raw에서는 후보가 많아지면 비슷한 문서 조각이나 주변 노이즈가 같이 들어오는 듯하다.
- 평가 자체에서도 버그가 있었다. 처음에는 답변 안에 "없습니다"가 있으면 무조건 abstention으로 봐서 q3/q4 같은 정상 답변을 거절로 오분류했다. 반대로 q10은 근거 목록이 길어서 "확인할 수 없습니다"가 정규식 범위 밖으로 밀렸다. 판정식을 고치고 저장된 detail JSON으로 지표를 재계산했다. 이 과정도 실험의 일부였다. 평가기가 틀리면 개선 방향도 틀어진다.

고민 / 실수

- 처음 raw 실험 결과에서 groundedness가 낮거나 답변이 비는 걸 보고 검색 문제라고 생각할 뻔했다. 실제 원인은 generator의 토큰 한도였다. 검색 지표, 답변 문자열, 응답 메타데이터를 같이 봐야 했다.
- abstention 판정도 생각보다 까다로웠다. "없는 요구사항을 없다고 답하는 것"과 "관련 문서를 못 찾아 답을 거절하는 것"은 다르다. 이 둘을 분리하지 않으면 환각방지 지표가 엉망이 된다.
- 점수가 높다고 바로 좋은 답변이라고 볼 수는 없었다. judge score가 비어 있거나, 답변이 길어서 사람이 읽기 부담스러운 케이스는 따로 확인해야 한다. 그래서 `missing_llm_judge_score`, `coverage_below_target`, `high_score_long_answer_needs_human_read` 같은 플래그를 남겼다.

추가 질문셋 준비

- 다음 평가를 위해 새 질문 후보 25개를 만들었다. 구성은 단일 추출 4개, 후속질문 4개, 비교 4개, 정확값 확인 4개, 모호한 기관명 3개, abstention 3개, score trap 3개다.
- 바로 사용하지 않고 JSON 파싱과 원본 파일명 매칭을 확인했다. 이 과정에서 질문 후보 JSON에 UTF-8 BOM이 붙어 일반 `utf-8` 파서가 실패하는 작은 문제를 발견했다. 내용은 정상이라 `utf-8-sig` 로 읽은 뒤 최종 `questions_v2.json` 은 BOM 없는 UTF-8로 다시 저장했다.
- abstention 질문을 제외한 target org는 모두 `data/원본 데이터/files` 아래 실제 파일명과 매칭됐다. 최종적으로 25개 모두 통과해서 `eval/questions_v2.json` 으로 고정했다.
- 평가 스크립트는 기존 `eval/questions.json` 에 고정되어 있었기 때문에, 질문 파일 경로를 바꿔 실행할 수 있도록 인자를 추가했다. 성능을 바꾸는 변경은 아니고, 같은 평가 엔진을 다른 고정 질문셋에 적용할 수 있게 한 배관 변경이다.

v2 질문셋 확인

- 새 질문셋이 실제 RAG 경로에서 동작하는지 1문항으로 확인했다. 질문은 `qv2_001_single_elec_security`, 한국전기안전공사 전기안전 관제시스템 보안 모듈 개발 용역 관련 질문이다.
- 결과는 coverage **1.0**, first_hit_rank **1**, abstention **false**, empty answer **false**, latency **24.93초**였다. 검색된 기관도 모두 한국전기안전공사였다.
- 답변도 직접 봤다. 보안모듈 적용, 통신서버 구성, 원격점검장치 요구사항을 구분했고, KCMVP, MQTT Broker, TLS, 원격점검장치 같은 핵심 근거가 문서 번호와 함께 나왔다. 새 질문셋이 실제 평가에 들어갈 수 있다는 최소 확인은 마쳤다.

대화 기반 인사이트 / 방향 재정리

- 지금까지의 coverage, MRR, relevance가 너무 높게 나와서 지표가 포화된 상태에 가깝다. 이 상태에서 검색 파라미터만 계속 바꾸면 실제 개선이 아니라 노이즈를 따라갈 수 있다. 성능을 더 올리려면 먼저 더 어려운 질문으로 측정계를 날카롭게 만들어야 한다.
- 특히 기존 평가셋에서 coverage/MRR 1.0, relevance 5.0이 반복되면 “시스템이 완벽하다”는 뜻이 아니라 “평가셋이 쉬울 수 있다”는 신호로 봐야 한다. 그래서 멀티홉, 비슷하지만 없는 사업, 애매한 발주기관, 수치 정확도, 사용자가 대충 묻는 질문을 일부러 넣어야 한다.
- 질문 생성은 클라우드 API를 꼭 쓸 필요가 없다. 오히려 로컬 CLI 기반으로 사람 같은 질문, 멍청한 질문, 진상스러운 질문, 오타 섞인 질문을 만드는 쪽이 과제 목적에 더 맞다. 답변 생성은 RAG 시스템이 맡고, 질문 생성과 원인분석은 별도 도구로 분리하는 편이 자연스럽다.
- 답변 채점은 자기 답변을 비슷한 모델이 다시 칭찬하는 구조가 되면 점수가 부풀 수 있다. answerer와 judge를 분리하고, groundedness/relevance 외에 contextual quality, evidence fit, usefulness, conciseness, wrong-document risk를 따로 봐야 한다.
- 정성 감리는 한 번에 큰 묶음으로 처리하면 중간 결과가 남지 않고 실패하기 쉽다. 앞으로는 케이스당 호출, 제한시간, 즉시 저장, 작은 묶음 검토 방식으로 바뀌어야 한다.
- 검색 품질은 이미 많이 올라왔기 때문에 남은 실질 개선 여지는 답변 지연시간, 답변 간결성, 정확값 질문 처리, 사업별 혼동 방지, 의미 기반 청킹 쪽에 있다.
- 같은 질문셋에 계속 맞추면 과적합이 생긴다. 다음부터는 튜닝용 질문과 검증용 질문을 나누고, 최종 성능은 튜닝에 쓰지 않은 질문으로 보고해야 한다.

먼저 처리한 준비 작업

- 답변 모델과 채점 모델을 분리할 수 있도록 설정을 추가했다. 앞으로는 RAG가 답한 모델과 답변을 평가한 모델을 결과에 따로 남길 수 있다.
- `questions_v2.json`을 튜닝용 18개와 검증용 7개로 나눴다. 검증용 질문은 최종 확인 전까지 개선 튜닝에 쓰지 않기로 했다.
- 정성 감리는 큰 묶음으로 한 번에 보지 않고 케이스별로 평가하고 바로 저장하는 방식으로 바꿨다. 1개 고위험 케이스를 먼저 확인했더니 `pass_with_caveat`가 나왔고, 자동점수만으로는 놓칠 수 있는 `coverage_below_target`, `context_mismatch` 위험이 같이 기록됐다.

- 현재 기본 설정만 따로 평가할 수 있는 실행 경로를 만들었다. 다음에는 이 경로로 v2 튜닝셋 전체 기준선을 먼저 잡는다.
- 이후 v2 튜닝셋 18문항 전체를 분리된 채점 기준으로 실행했다. 결과는 EDD **79.90**, coverage **0.911**, MRR **1.000**, groundedness **3.125**, relevance **4.812**, abstention accuracy **0.500**, latency **17.922초**였다.
- 이 결과는 오히려 좋은 신호다. 이전처럼 1.0/5.0에 포화되지 않고, 실제 개선할 지점이 드러났다. 특히 검색은 첫 정답 순위가 좋지만 답변이 근거 없는 세부사항을 섞어 groundedness가 떨어지는 문제가 보였다.
- 대표적으로 한국전기안전공사 질문은 coverage 1.0, rank 1, relevance 5였지만 groundedness 2였다. 답변은 질문 범주에는 잘 맞았지만, 일부 버전명과 세부 기능이 제공 근거에서 확인되지 않는다는 지적이 나왔다. 검색 성공이 곧 답변 근거성 성공은 아니라는 것을 확인했다.
- 비교 질문에서는 여러 기관을 모두 잡지 못하는 coverage 문제가 남아 있었고, 없는 정보 질문에서는 abstention 판단도 더 정교하게 나눠야 했다. 이제 개선 방향은 검색 파라미터보다 근거 없는 세부사항 억제, 비교 질문 multi-org 보강, 정확값 미기재 처리, 답변 간결성 쪽이다.
- 문항별 실패 분석도 따로 만들었다. 18문항 중 이슈가 잡힌 케이스는 13개였고, 유형별로는 근거성 부족 10개, 비교/모호기관 검색 실패 3개, 정확값 미기재와 거절 판정이 섞인 케이스 1개, 없는 정보를 끝까지 거절하지 못한 케이스 1개였다.
- 이 분석 때문에 다음 순서가 더 분명해졌다. 검색은 이미 꽤 잘 맞고 있으니 무작정 후보를 늘리는 것보다, 답변이 근거에 없는 버전명·세부 기능·요구사항 번호를 덧붙이지 못하게 막는 것이 먼저다. 그 다음 비교 질문에서 여러 기관을 빠뜨리지 않는 검색 보강, 정확값 질문의 평가 기준 분리, 없는 정보 질문의 거절 강화로 간다.

EDD를 더 끌어올리기 위한 추가 루프

- 처음에는 근거성 부족 10개를 보고 답변 프롬프트가 가장 큰 문제라고 생각했다. 그래서 근거 문서 번호를 강제로 붙이는 프롬프트, 확인된 내용만 짧게 쓰는 프롬프트를 따로 비교했다. 그런데 기대만큼 점수가 오르지 않았다.
- 뜯어보니 더 큰 원인이 있었다. 답변을 평가하는 쪽이 검색된 근거 전체를 보는 게 아니라 앞부분 위주로만 보고 있었다. 그래서 뒤쪽에 있는 BioIN, 비교 대상 기관, 추가 근거를 못 본 채 "근거 없음"으로 깎는 일이 생겼다. 이건 답변 문제이기도 하지만, 더 정확히는 평가 방식 문제였다.
- 평가기가 각 문서 조각의 사업명·발주기관·본문 일부를 균등하게 보도록 바꾸자, 대표 3문항의 groundedness가 1.667에서 5.0으로 회복됐다. 전체 18문항에서도 EDD가 **79.90 → 84.79**로 올랐다. 교훈은 분명했다. 성능을 올리려면 모델을 고치기 전에, 평가기가 정답 근거를 제대로 보고 있는지부터 확인해야 한다.
- 그 다음 남은 문제는 비교 질문이었다. 한영대학·전북대·대전대·울지대처럼 4개 기관을 비교하는 질문인데, 기존 기관 필터가 3개까지만 잡아 한 기관을 빠뜨렸다. 또 "봉화군", "충북연구원", "광주연구원", "광주문화재단"처럼 사용자가 줄여 부르는 기관명은 정식 발주기관명과 안 맞아 누락됐다.
- 발주기관명에서 **재단법인**, **광역시·도** 접두어를 떼어 별칭으로도 찾게 하고, **대학교/대학** 표기 차이도 허용했다. 비교 질문에서는 최대 6개 기관까지 잡도록 넓혔다. 그 결과 비교/모호기관 대표 3문항은 coverage가 모두 1.0으로 회복됐다.

- 없는 정보 질문도 다시 봤다. qv2_021은 실제 답변에서 "최종 낙찰 업체명, 최종 계약금액, 제안서 평가점수는 문서에서 확인할 수 없다"고 잘 답했는데, 평가기가 이 문장을 거절 답변으로 못 잡고 있었다. 그래서 "요청하신 항목이 확인되지 않는다"는 형태와 최종 낙찰·계약금액·평가점수가 반복적으로 확인 불가인 형태를 거절로 잡도록 보정했다.
- 세 보정을 합쳐 다시 18문항 전체를 돌렸더니 EDD가 **95.55**까지 올랐다. 세부 지표는 coverage **1.0**, hit-all **1.0**, MRR **1.0**, groundedness **4.938**, relevance **4.938**, abstention accuracy **1.0**, false abstention **0.0**, empty answer **0.0**, latency **25.413초**였다.
- 속도를 더 줄이기 위해 top_k를 5로 낮춘 후보도 시험했다. 재계산 기준 EDD는 **96.34**까지 나왔다. 하지만 qv2_003, qv2_009, qv2_017에서 근거성이나 관련성이 조금 낮아졌다. 숫자만 보면 더 좋지만, 실제 답변 품질이 일부 흔들리는 후보라 바로 최종 채택하지 않았다.
- 지금 판단은 이렇다. 안정적인 최종 후보는 EDD **95.55**인 top_k 8 설정이고, 더 높은 점수를 노릴 후보는 EDD **96.34**인 top_k 5 설정이다. 다만 과제 목적상 숫자만 높은 후보를 바로 쓰면 안 되고, top_k 5가 만든 답변 품질 하락을 정성 감리로 확인해야 한다. 다음 개선은 어려운 질문은 top_k 8, 쉬운 질문은 top_k 5로 가는 적응형 설정을 실험하는 쪽이다.

아직 숫자를 믿기 전에 확인해야 할 것

- 다시 생각해보니 EDD **95.55**는 강한 후보 점수이지, 아직 최종 성능이라고 말하면 안 된다. 이유는 두 가지다.
- 첫째, 평가기가 근거를 균등하게 보도록 바꾸면서 groundedness가 크게 올랐다. 이 변화는 앞부분만 보던 평가를 바로잡은 면이 분명 있지만, 동시에 평가기가 더 후해진 효과일 수도 있다. 그래서 같은 답변을 두 방식으로 다시 채점해 봐야 한다. 앞부분만 보는 방식에서는 왜 낮았고, 균등하게 보는 방식에서는 왜 높았는지 답변별로 비교해야 한다. 그래야 "진짜 답변이 좋아졌다"와 "채점 방식이 바뀌었다"를 분리할 수 있다.
- 둘째, 18문항 튜닝셋에서 다시 지표가 거의 천장에 닿았다. coverage, hit-all, MRR, abstention이 모두 1.0이고 groundedness/relevance도 4.938이면 좋아 보이지만, 같은 질문셋에 맞춰진 결과일 가능성도 있다. 그래서 최종 주장은 아직 보지 않은 7문항 검증셋으로 확인해야 한다.
- 검증셋 실행을 시작하려고 했지만, 이 비판 내용을 먼저 기록해야 한다는 판단으로 중단했다. 따라서 검증셋 결과는 아직 없다. 이 상태에서 보고서에는 "95.55까지 개선됐다"가 아니라 "튜닝셋 기준 95.55까지 개선됐고, 채점 방식 검증과 검증셋 확인이 다음 필수 단계"라고 써야 한다.
- 속도도 남은 문제다. top_k 8은 안정적이지만 평균 **25.413초**로 느리고, top_k 5는 **17.494초**까지 빨라지지만 일부 어려운 문항에서 답변 품질이 떨어졌다. 속도 개선은 필요하지만, 품질을 잃고 얻은 점수 상승은 채택하면 안 된다.

검증셋 확인과 다시 개선한 내용

- 위의 반박 관점을 기록한 뒤, 실제로 아직 보지 않았던 7문항 검증셋을 돌렸다. 처음 결과는 EDD **81.55**였다. 이 결과가 중요했다. 튜닝셋에서는 **95.55**까지 올랐지만, 검증셋에서는 그대로 유지되지 않았다. 즉 "좋아 보이는 숫자"를 바로 최종 성능으로 쓰면 위험하다는 지적이 맞았다.
- 실패는 크게 세 가지였다. 첫째, 서울·인천·부산 플랫폼 비교 질문에서 부산관광공사 문서가 빠져 coverage가 **0.667**로 떨어졌다. 둘째, 고려대·광주과기원·한영대 성능기준 비교 질문에서 광주과학기술원 문서가 빠져 coverage가 **0.667**이었다. 셋째, 투입 개발자 실명·개인 휴대전화·개인 이메일

을 묻는 질문은 답변이 실제로 “문서에서 확인할 수 없다”고 반복했는데도, 평가기가 이를 거절 답변으로 잡지 못했다.

- 원인은 분명했다. 여러 기관을 필터로 잡아도 최종 근거 8개를 고르는 과정에서 특정 기관이 후보 밖으로 밀릴 수 있었다. 그래서 여러 기관이 명시된 질문에서는 필터에 잡힌 기관마다 최소 1개 근거가 들어오도록 보강했다. 후보 풀에 아예 없으면 해당 기관만 따로 1개 더 찾아 넣는 방식이다.
- 개인정보 질문은 답변 문구가 조금 달라지면 평가기가 놓쳤다. 앞에서는 “투입 개발자 실명·개인 휴대전화·개인 이메일” 같은 필드명이 반복될 때만 잡았는데, 이번 답변은 사업명마다 “제공된 문서에서 확인할 수 없습니다”만 반복했다. 그래서 이런 전면 미확인 답변도 거절로 잡도록 보정했다.
- 수정 후 실패 3문항만 먼저 확인했더니 coverage **1.0**, hit-all **1.0**, abstention accuracy **1.0**으로 회복됐다. 그 다음 7문항 검증셋 전체를 다시 확인했고, 새 기준으로 재계산한 결과 EDD **96.80**이 나왔다. 세부 지표는 coverage **1.0**, hit-all **1.0**, MRR **1.0**, groundedness **4.833**, relevance **5.0**, abstention accuracy **1.0**, latency **19.127초**였다.
- 중간 실수도 있었다. 실패 3문항만 빠르게 확인하려다가 질문 파일 생성과 실행을 잘못 묶어 파일 없음 오류가 났고, 누락 기관을 추가로 넣는 코드에서는 Chroma가 돌려준 임베딩 배열에 바로 append를 하려다 타입 오류가 났다. 둘 다 성능 결과로 쓰지 않고 원인만 남긴 뒤, 파일을 따로 만들고 배열을 리스트로 바꾸어 다시 확인했다.
- 다만 완전히 끝난 것은 아니다. 검증셋에서 qv2_008은 groundedness **4**였다. 고시 개정 반영 범위와 기존 시스템 보완 범위를 잘 나눴지만, 일부 세부 표현이 문서에 직접 보이는 근거보다 조금 넓게 쓰였다는 지적이 있었다. 튜닝셋 쪽 qv2_003도 비슷하게 다른 문서의 보안 조항이 섞이는 위험이 남아 있다.
- 채점 방식 검증도 했다. 같은 저장 답변을 앞부분 근거만 보는 방식과 문서 조각을 균등하게 보는 방식으로 다시 채점했다. 7개 대표 문항에서 앞부분 방식의 groundedness 평균은 **2.0**, 균등 방식은 **4.714**였다. 관련성 평균은 둘 다 **4.857**이었다. 엄격하게 “근거 없는 구체 주장”만 찾는 검토에서는 평균 위험도가 **0.0**으로 나왔다.
- 이 결과는 균등 방식이 그냥 후해졌다고만 보기는 어렵다는 뜻이다. 실제로 앞부분 방식은 뒤쪽 근거를 못 봐 낮게 준 경우가 많았다. 다만 qv2_003은 균등 방식에서도 groundedness **3**, relevance **4**로 낮게 남았다. 그래서 측정 보정은 대체로 타당하지만, 특정 케이스의 근거 범위 오염은 아직 실제 문제로 남아 있다고 보는 게 맞다.
- 속도 후보도 다시 검토했다. top_k 5는 빠르지만 qv2_003, qv2_009, qv2_017 같은 어려운 질문에서 품질이 흔들렸다. 그래서 전역 기본값으로 낮추는 것은 보류한다. 다음에는 쉬운 단일 질문이나 정확값 질문만 top_k 5를 쓰고, 비교·모호기관·후속질문·없는 정보 질문은 top_k 8을 쓰는 적응형 방식으로 실험하는 것이 맞다.

다음 액션

- qv2_003과 qv2_008처럼 다른 문서의 보안 조항이나 주변 설명이 섞이는 케이스를 줄인다.
- 답변에서 각 bullet이 어느 사업/문서 범위에 속하는지 더 엄격히 제한하는 citation guard를 실행한다.
- 쉬운 질문은 top_k 5, 어려운 비교·후속·없는정보 질문은 top_k 8로 보내는 적응형 설정을 시험한다.

- 새 설정은 튜닝셋뿐 아니라 검증셋에서도 다시 확인한다.
- 이후 사람 같은 질문을 더 추가해 오타, 대중 부르는 기관명, 여러 사업 섞기, 없는 정보 우기기, 짧고 무례한 요구, 후속질문을 포함한 새 질문셋을 만든다.

소감

- 오늘은 숫자를 올리는 것보다, 숫자를 믿어도 되는지 확인하는 장치를 만드는 데 더 가까웠다. EDD가 높아도 답변이 길거나 맥락을 빗나갈 수 있고, 평가기가 틀리면 개선 방향도 틀어진다. 이제 보고서에 “성능이 올랐다”만 쓰는 게 아니라, 어떤 질문에서 좋아졌고 어떤 질문은 왜 조심해야 하는지까지 말할 수 있게 됐다.

검증 숫자 라벨을 다시 바로잡은 내용

- 다시 보니 더 엄격한 비판이 맞았다. 순서는 분명했다. 먼저 튜닝셋에서 EDD **95.55**가 나왔고, 처음 7문항 검증셋에서는 EDD **81.55**로 떨어졌다. 그 다음에야 어떤 3문항이 틀렸는지 확인했고, 그 실패를 보고 검색 보강과 거절 판정 규칙을 고쳤다. 그 뒤 같은 7문항을 다시 재서 EDD **96.80**이 나온 것이다.
- 그래서 **96.80을 엄밀한 검증셋 성능이라고 쓰면 안 된다**. 이 숫자는 “같은 검증셋의 실패를 보고 고친 뒤 다시 잰 값”이다. 보고서에는 정직하게 튜닝셋 95.55 / 첫 검증셋 81.55 를 일반화 경고로 쓰고, 96.80 은 같은 문항 재측정이라는 꼬리표를 달아야 한다.
- 7문항이라는 크기도 작다. 두세 문항이 뒤집히면 EDD가 크게 출렁인다. 따라서 96.80 같은 소수점 숫자를 아주 정밀한 최종 성능처럼 말하면 오히려 신뢰를 잃는다. 이번 과제에서 더 좋은 태도는 “숫자가 올랐다”보다 “어떤 절차 때문에 그 숫자를 어디까지 믿을 수 있는가”를 밝히는 것이다.
- 채점 방식도 아직 완전히 믿으면 안 된다. 문서 조각을 균등하게 보여주는 방식으로 바꾸니 groundedness가 크게 올랐는데, 이는 실제 답변이 좋아진 효과와 채점 방식이 바뀐 효과가 섞인 결과다. 특히 qv2_003처럼 다른 문서의 보안 조항이 섞이는 케이스가 남아 있는데도, 엄격 검토에서 위험도 0.0이 나온 점은 채점기가 너무 느슨한지 다시 봐야 한다. 일부러 틀린 답변을 넣어 잡아 내는지 확인하는 검사가 필요하다.
- 요약표도 손봤다. 채점 없이 빠르게 확인한 행이 groundedness/relevance 0.0으로 들어가 EDD **56.89, 49.95**처럼 보였는데, 이건 성능 하락이 아니라 품질 채점이 끝나지 않은 진단용 행이었다. 이제 이런 행은 순위와 그래프에서 빼고, 별도 진단 행으로만 남기도록 정리했다.
- 기관별 근거 보강도 조심해야 한다. 여러 기관 비교 질문에서 빠진 기관을 채워 넣는 보강은 이번 실패를 잘 고쳤지만, 기관명이 잘못 잡히면 엉뚱한 문서를 억지로 넣을 위험이 있다. 다음에는 비슷한 이름의 기관, 줄여 부르는 기관명, 실제로는 없는 조합을 넣어 오탐이 생기는지 따로 봐야 한다.
- 오늘의 결론은 꽤 냉정하다. 성능은 실제로 좋아졌지만, 가장 정직한 일반화 증거는 아직 **81.55**다. 96.80은 개선 가능성을 보여주는 재측정값이지, 새 검증셋에서 확인된 최종 성능은 아니다. 다음에는 새로 건드리지 않은 검증 문항을 만들고, 일부러 틀린 답변을 넣어 채점기가 잡는지 본 다음, 그 위에서 지연시간과 답변 간결성을 손봐야 한다.

시간 기준 개선 루프 운영 인사이트

- 무한히 고치는 방식은 보고서에도 실험에도 좋지 않다. 그래서 앞으로는 “몇 시까지”라는 기준을 두고 움직이되, 실행 중인 작업을 억지로 끊지는 않기로 했다. 시간은 칼같이 자르는 선이 아니라, 새 일을 더 벌릴지 말지 판단하는 기준선으로 쓴다.
- 예를 들어 20시까지로 잡았다면, 20시 직전에는 새 큰 평가를 시작하지 않는다. 이미 답변 생성이나 채점이 돌고 있다면 현재 단위는 마무리하고, 결과 파일과 원인 분석을 남긴 뒤 닫는다. 만약 22시까지 흘러도 현재 단위를 끝내는 편이 더 안전하면, 그 단위까지만 끝내고 다음 가설은 다음 차례로 넘긴다.
- 매 차례의 기본 흐름은 `가설 선택 → 질문 고정 → 답변 생성 → 채점 → 낮은 품질 답변 확인 → 좁은 수정 → 재측정 → 비판 검토 → 점수 라벨링 → 일지 기록`으로 잡는다. 특히 마지막 비판 검토를 빼면, 숫자가 오른 이유가 실제 개선인지 착시인지 흐려진다.
- 질문자는 매번 달라져야 한다. 같은 질문을 반복하면 시스템이 그 문항에만 익숙해진다. 오타, 줄임말, 대충 부르는 기관명, 없는 정보 우기기, 짧고 무례한 요구, 후속질문을 섞어 “사람이 실제로 물어볼 법한 질문”을 계속 새로 만들어야 한다.
- 품질 낮은 답변은 버리지 않고 다시 본다. 다만 같은 질문을 보고 고친 뒤 오른 점수는 일반화 점수가 아니라 재시도 점수다. 보고서에는 처음 본 질문 점수, 실패를 보고 고친 재시도 점수, 새 검증 셋 점수를 반드시 분리해서 쓴다.
- 외부 검토 도구를 붙이는 것도 좋다. 다만 외부 검토 결과도 권위가 아니라 증거로 봐야 한다. 외부 검토가 “이 점수는 오염됐다”, “채점자가 후하다”, “답변은 길지만 질문에 덜 맞다”고 지적하면, 그 지적을 다시 실험 가설로 바꾸고 다음 차례에서 확인한다.
- 이 방식이면 실험을 많이 하면서도 끝없이 새는 것을 막을 수 있다. 중요한 것은 마지막에 “오늘 무엇이 좋아졌는가”만 쓰는 게 아니라, “어떤 숫자는 믿을 수 없게 되었는가”까지 함께 쓰는 것이다. 그 기록이 다음 개선의 출발점이 된다.

루프 1차: 새 질문셋과 지점별 비교

- 18시 25분에 20시를 기준선으로 잡고 새 루프를 시작했다. 이번에는 질문 생성에 추가 비용을 쓰지 않고, 실제 원본 파일명과 지금까지의 실패 유형을 보고 직접 새 검증 문항을 만들었다. 오타 섞인 질문, 후속질문, 같은 기관의 여러 사업 비교, 없는 정보 우기기, 개인정보 요구를 섞어 12문항으로 고정했다.
- 실행 전에 12문항 모두 JSON 파싱과 원본 파일명 매칭을 확인했다. 이 단계에서 문항 자체가 잘못되어 점수가 흔들리는 일을 막고 싶었다. 새 문항은 기존에 실패를 보고 고친 7문항이 아니므로, 첫 실행 점수는 새 검증 근거에 가깝게 볼 수 있다.
- 첫 지점(L0)은 top_k 8 현재 설정 그대로 실행한 결과다. EDD는 **89.69**, coverage/hit-all/MRR/groundedness/relevance는 모두 **1.0 또는 5.0**이었는데, abstention accuracy가 **0.333**으로 낮았다. 처음엔 “없는 정보 질문을 못 막았나?” 싶었지만 답변을 읽어보니 두 문항은 실제로 “문서에서 확인할 수 없다”고 잘 답했다. 문제는 답변이 아니라 거절 판정식이었다.
- 그래서 두 번째 지점(L1)은 같은 답변을 다시 생성하지 않고, 저장된 답변에 대해 거절 판정식만 보정해 재계산했다. 서민금융진흥원 최종 낙찰/계약금액/개인 연락처 문항과 네팔 Pilot 사업의 투입 개발자 개인정보 문항이 새로 거절로 잡혔다. EDD는 **96.36**으로 올라갔다. 이 숫자는 새 답변 성능이 아니라 **측정 보정 후 같은 답변 점수로** 라벨을 붙여야 한다.

- 그 다음 속도를 보려고 top_k 5 후보도 확인했다. 세 번째 지점(L2)의 원점수는 **91.41**, 보정 후 네 번째 지점(L3)은 **94.74**였다. 평균 지연시간은 **24.021초 → 21.372초**로 약 **2.65초** 줄었지만, groundedness/relevance가 **5.0/5.0 → 4.667/4.778**로 내려갔다.
- 특히 안양 호계체육관 예약시스템 문항에서 차이가 컸다. top_k 8은 예약·결제·회원 통합운영을 잘 잡았는데, top_k 5는 결제/PG 연동을 문서에서 확인할 수 없다고 잘못 말했다. 이 지점이 “그 다음 한 일은 과했다”고 볼 수 있는 부분이다. 속도를 조금 얻었지만, 답변 품질을 잃었다.
- 그래서 현재 최선 지점은 L1이다. 다만 L1도 “새 답변 실행”이 아니라 측정 보정 점수이므로, 보고서에서는 L0 **89.69**를 첫 원점수로, L1 **96.36**을 같은 답변 재계산 점수로 나눠 써야 한다. top_k 5는 전역 기본값으로 채택하지 않는다.
- 이번 루프의 교훈은 분명하다. 질문셋을 새로 만들면 다시 약점이 보이고, 루프 지점을 나눠 표시하면 어느 순간부터 개선이 아니라 과잉 튜닝이 되는지도 보인다. 오늘은 더 많은 실행을 억지로 넣는 것보다, 이 네 지점의 의미를 정리하고 다음에는 채점기 보정 검증이나 근거 범위 오염 방지로 가는 편이 낫다.

루프 1차: L4 검색 범위 보정

- 위 네 지점을 다시 보다가 안양 호계체육관 문항을 그냥 넘기면 안 된다고 판단했다. top_k 5가 틀린 이유를 단순히 “문서 조각이 적어서”라고만 쓰기에는 부족했다. 저장된 근거 기관 목록을 보니 안양시 문서만 들어간 것이 아니라 고양, 정읍, 대한장애인체육회 쪽 체육시설 문서도 같이 섞여 있었다. top_k 8에서는 필요한 조각이 뒤쪽에 남아 답변이 살아났지만, top_k 5에서는 그 조각이 빠지면서 결제/PG 연동을 없다고 말해 버린 것이다.
- 그래서 다섯 번째 지점(L4)은 새 답변이나 새 채점이 아니라, 검색 범위가 흔들린 원인을 좁히는 작업으로 잡았다. 질문에는 “안양 호계체육관”이라고 되어 있었고 “안양시”라고 정확히 쓰지는 않았다. 기존 규칙은 정식 발주기관명에는 강했지만, 사용자가 도시 이름만 짧게 부르는 경우에는 기관 필터가 충분히 켜지지 않았다.
- 발주기관명에서 도시 접두어를 걷어낸 뒤, 안양, 평택, 봉화, 정읍 처럼 짧게 부르는 행정구역 이름도 별칭으로 잡도록 보정했다. 또 전북특별자치도 정읍시 처럼 새 행정명 접두어도 빠져 있어서 함께 추가했다.
- 비용이 드는 새 답변 생성은 하지 않고, 기관 필터만 확인했다. 결과는 안양 → 경기도 안양시, 평택 → 경기도 평택시, 봉화 → 경상북도 봉화군, 정읍 → 전북특별자치도 정읍시 로 나왔다. 즉 L4는 EDD가 붙는 성능 지점이 아니라, 다음 실행에서 엉뚱한 체육시설 문서가 섞일 가능성을 줄이는 준비 지점이다.
- 이 지점까지 보고 나니 오늘 더 큰 실행을 욕여넣는 것은 좋지 않다고 봤다. 현재 최선으로 믿을 수 있는 지점은 여전히 L1, EDD **96.36**이다. L2/L3은 속도를 조금 얻었지만 품질을 잃은 지점이고, L4는 점수 상승 주장이 아니라 원인 제거다. 다음에 한다면 qv3_010만 작게 다시 확인하거나, 일부러 틀린 답변을 넣어 채점기가 잡는지 보는 검증으로 가야 한다.

루프 1차: L5 별칭 오탐 방지

- L4를 하고 나니 바로 반대쪽 걱정이 생겼다. 짧은 지명을 잘 잡게 만들면, 이번에는 잡지 말아야 할 단어까지 잡을 수 있다. 그래서 새 답변을 만들기 전에 “안양대학교”, “평택대학교”, “봉화산역”, “안양천”처럼 발주기관으로 단정하면 안 되는 문구를 넣어 봤다.

- 결과는 예상대로 위험했다. 처음에는 안양대학교가 경기도 안양시 로, 평택대학교가 경기도 평택시 로, 봉화산역이 경상북도 봉화군 으로, 안양천이 경기도 안양시 로 잡혔다. 이 상태에서 실제 사용자가 대학이나 하천을 물으면 엉뚱한 발주기관 문서를 억지로 끌고 올 수 있었다.
- 그래서 짧은 지명 뒤에 대학교, 대학원, 대학, 산역, 역, 천 같은 접미사가 붙으면 행정구역 별칭으로 보지 않도록 막았다. 다시 확인하니 네 오답 문구는 모두 필터가 걸리지 않았고, 원래 살려야 했던 안양 호계체육관, 평택 버스정보시스템, 봉화 스마트팜, 정읍체육트레이닝센터 는 그대로 정확한 기관으로 잡혔다.
- 이 여섯 번째 지점(L5)도 EDD 점수는 없다. 대신 이전 보정이 만든 부작용을 바로 공격해서 줄였다는 의미가 있다. 오늘의 좋은 흐름은 이 부분이다. 점수가 오른 뒤에 끝내지 않고, 그 점수를 올린 보정이 다른 질문에서 망가질 가능성을 다시 확인했다.
- 따라서 20시 전 남은 시간에 더 큰 평가를 새로 시작하지 않는 쪽이 맞다. 지금은 L0~L5의 의미가 충분히 생겼고, 다음 차례는 새 비용을 쓰는 큰 실행보다 블라인드 채점 검증이나 qv3_010 소규모 재확인처럼 목적이 분명한 작업이어야 한다.
- 마지막으로 L0~L5를 한눈에 볼 수 있게 지점 그래프도 남겼다. 그래프에서는 EDD가 있는 L0~L3만 막대로 보이고, L4/L5는 점수 없는 진단 지점으로 따로 표시된다. 이렇게 해 두면 나중에 “어느 지점이 제일 좋았고, 어디부터 과했는지”를 숫자와 절차를 같이 보며 판단할 수 있다.

루프 1차: L6 안양 문서 안의 근거 확인

- 다음으로 바로 새 답변을 돌리기 전에, 정말 안양 문서 안에 결제/PG/회원 근거가 있는지부터 봤다. 만약 원문 안에 근거가 없다면 재실행을 해도 답변이 좋아질 이유가 없기 때문이다. 그래서 답변 생성이나 채점 없이, 고쳐진 기관 필터로 잡히는 저장 문서 조각만 직접 확인했다.
- 결과는 분명했다. 안양 호계체육관 질문은 경기도 안양시 로 잡혔고, 그 범위 안에는 호계체육관 배드민턴장 및 탁구장 예약시스템 구축 용역 조각 106개가 있었다. 그 안에서 예약은 39회, 결제는 7회, PG는 2회, 회원은 8회, 키오스크는 13회, 발권은 3회, 매출은 4회, 무인은 5회 확인됐다.
- 특히 핵심 조각에는 “외부 오프라인 PG 모듈을 이용하여 예약 및 결제관리”, “회원 및 비회원 입장객 입장에 대한 매출등록 및 결제”, “키오스크 시스템 기능 요구사항”이 같이 들어 있었다. 즉 이전 top_k 5 답변이 “결제/PG를 확인할 수 없다”고 한 것은 원문에 근거가 없어서가 아니라, 질문 범위가 다른 체육시설 문서와 섞이고 필요한 조각이 빠진 결과로 보는 것이 맞다.
- 이 일곱 번째 지점(L6)도 EDD 점수는 없다. 하지만 다음 행동을 더 명확하게 만든다. 이제 qv3_010을 작게 다시 돌린다면, 목적은 “고친 필터가 실제 답변에서 결제/PG/회원 요구를 회복시키는가”를 보는 것이다. 반대로 전체 질문셋을 다시 크게 돌리는 것은 아직 이르다.

루프 1차: L7 채점기 검증팩 준비

- 다음에는 채점기를 바로 믿지 않기 위한 준비를 했다. 지금까지 계속 나온 걱정은 “점수가 높아도 채점자가 후하면 착시일 수 있다”는 것이었다. 이 문제를 말로만 남기면 다음에 또 흐려질 수 있으니, 일부러 맞는 답변과 틀린 답변을 섞은 검증팩으로 고정했다.
- 검증팩에는 여섯 가지 함정을 넣었다. 첫째, 안양 호계체육관 문서 근거만으로 짧고 정확하게 답한 정상 답변. 둘째, 근거에 결제/PG가 있는데도 없다고 말하는 답변. 셋째, 안양 문항에 고양이나 정읍 쪽 체육시설 내용을 섞은 답변. 넷째, 최종 낙찰업체·계약금액·개인 연락처를 확인할 수 없다고

거절하는 정상 답변. 다섯째, 낙찰업체와 개인 연락처를 지어내는 답변. 여섯째, 수자원공사 비교 질문에서 타당성 조사를 시스템 구축처럼 말하는 답변이다.

- 이 지점(L7)도 새 점수는 없다. 대신 다음에 강한 채점기를 쓸 때 “이 답변은 반드시 떨어뜨려야 한다”는 기준을 미리 박아 둔 것이다. 만약 나중에 채점기가 이렇게 심어 둔 오답에도 높은 groundedness를 주면, 그때의 높은 EDD는 믿으면 안 된다. 먼저 채점 기준을 고쳐야 한다.
- 오늘 흐름을 보면 L1이 가장 좋은 점수 지점이고, L4~L7은 그 점수를 믿기 위한 안전장치에 가깝다. 숫자를 더 올리는 것보다, 숫자를 속이는 길을 하나씩 막는 쪽으로 정리되고 있다.

루프 1차: L8 채점기 검증 실행 준비 확인

- 마지막으로 L7에서 만든 검증팩을 실제로 읽고 결과 파일까지 남길 수 있는지 확인했다. 이번에도 채점 호출은 하지 않았다. 대신 기대값만 읽는 모드로 실행해서, 검증팩 6개가 정상적으로 잡히고 그중 통과 기대 2개, 실패 기대 4개로 정리되는지 봤다.
- 결과는 정상이다. 이제 나중에 비용을 써도 된다고 판단되는 시점에는 같은 검증팩을 그대로 사용해 채점기를 부를 수 있다. 중요한 점은 이 검사가 넓은 전체 평가가 아니라, 일부러 심어 둔 6개 함정만 보는 작고 통제된 검사라는 것이다.
- 여기까지 오면 오늘의 다음 판단은 더 또렷하다. 새 점수를 하나 더 얻으려고 큰 평가를 돌릴 필요는 없다. 남은 선택지는 두 가지다. 하나는 qv3_010만 작게 다시 답하게 해서 결제/PG가 회복되는지 보는 것, 다른 하나는 L7/L8 경로로 채점기가 심은 오답을 잡는지 보는 것이다.

루프 1차: L9/L10 안양 문항 표적 재확인

- 이제 실제 답변이 회복되는지 딱 한 문항만 확인했다. 전체 평가도 아니고 채점도 하지 않았다. 대상은 계속 문제가 됐던 안양 호계체육관 문항이다.
- 먼저 top_k 5로 다시 답하게 했다(L9). 검색된 5개 기관은 모두 경기도 안양시였다. 이 부분은 좋아졌다. 예전처럼 고양, 정읍, 다른 체육시설 문서가 섞이는 문제는 사라졌다. 하지만 답변을 읽어보니 여전히 PG를 직접 말하지 못했고, 결제수단이나 지불처리 방식은 확인할 수 없다고 썼다. L6에서 이미 PG 근거가 원문 안에 있음을 확인했기 때문에, 이것은 “원문에 없음”이 아니라 “top_k 5가 필요한 조각을 못 잡음”으로 보는 게 맞다.
- 그래서 같은 문항을 top_k 8로 한 번 더 확인했다(L10). 이번에도 검색 기관은 8개 모두 경기도 안양시였고, 답변에는 예약, 결제, PG, 회원, 키오스크, 발권, 매출이 모두 들어왔다. 특히 “외부 오프라인 PG 모듈 연동”, “발권기 설정”, “매출 정보 연동”까지 회복됐다.
- 이 비교로 결론이 더 단단해졌다. top_k 5는 오염을 줄인 뒤에도 세부 결제 근거를 놓칠 수 있다. 안양 호계체육관처럼 시설 예약, 회원, 결제, 발권, 매출이 함께 얹힌 질문은 top_k 8이 맞다. 속도를 위해 무조건 top_k 5로 낮추는 것은 오늘 결과로 다시 기각한다.
- 다만 L9/L10은 채점 없는 표적 확인이다. 따라서 여기서 나온 점수처럼 보이는 값은 성능 점수로 쓰면 안 된다. 보고서에는 “답변 내용 비교 결과 top8이 PG 세부근거를 회복했다”는 정성 결과로만 써야 한다.

루프 1차: L11 적응형 top_k 보호 규칙

- L9/L10에서 얻은 결론을 그대로 코드의 선택 규칙으로 옮겼다. 기본값은 건드리지 않았다. 지금 기본은 이미 top_k 8이라 안전하다. 다만 나중에 속도를 줄이려고 top_k 5 후보를 다시 쓰더라도, 시설 예약·결제·PG·회원·키오스크·발권·매출처럼 세부근거가 많은 질문은 자동으로 최소 top_k 8을 쓰도록 선택 규칙을 추가했다.
- 이 규칙은 기본으로 켜지지 않는다. 실험에서 따로 켜었을 때만 작동한다. 확인해 보니 안양 호계체육관처럼 예약·결제·PG가 있는 질문은 top_k 5 요청이어도 8로 올라갔고, 평택 버스정보시스템처럼 단순한 안내기 질문은 5로 유지됐다. 이 기능을 끄면 원래처럼 5로 남는다.
- L11도 성능 점수는 아니다. 하지만 L9/L10의 교훈을 다음 실험에서 반복해서 적용할 수 있게 만든 준비 작업이다. 다음에 실제 성능으로 주장하려면, 쉬운 질문은 5로 유지되고 세부근거 질문은 8로 올라가는 혼합 질문셋에서 다시 측정해야 한다.

답변품질 관점에서 아직 아쉬운 점

- 오늘 점수와 검색 범위는 많이 정리됐지만, 답변품질 자체에는 아직 아쉬움이 남아 있다. 첫째, 근거를 회복한 답변과 사람이 읽기 좋은 답변은 다르다. L10은 PG·발권·매출을 회복했지만 여전히 문서 나열식이고 길다. 평가자가 보면 “맞긴 한데 요약력이 부족하다”고 볼 수 있다.
- 둘째, 근거 표시가 아직 충분히 친절하지 않다. 답변에 문서 번호가 붙어도, 사용자는 그 문서가 어떤 사업의 어떤 요구사항인지 바로 알기 어렵다. 사업명, 요구사항 번호, 핵심 원문 조각이 함께 보여야 납득력이 높아진다.
- 셋째, 높은 점수를 아직 완전히 믿으면 안 된다. 함정 답변 검증팩은 만들었지만 실제 채점기가 그 오답을 잡아내는지는 아직 확인하지 않았다. 따라서 EDD가 높게 나오는 결과도 “채점기를 통과한 조건부 숫자”로 다뤄야 한다.
- 넷째, 적응형 top_k 규칙도 아직 후보일 뿐이다. qv3_010에서는 방향이 맞았지만, 쉬운 질문과 어려운 질문을 섞은 작은 묶음에서 실제로 지연시간과 품질을 동시에 잡는지 다시 봐야 한다.
- 그래서 다음으로 가장 좋은 작업은 대표 답변 6~8개를 골라 사람 관점의 품질 리뷰표를 만드는 것이다. 질문, 답변, 왜 좋은지, 왜 아쉬운지, 점수는 높지만 납득이 안 되는 부분, 보고서에 쓸 수 있는 인사이트를 한 줄씩 정리하면 보고서 설득력이 크게 올라간다.

대표 답변 품질 리뷰표 작성

- 위 아쉬움을 그냥 말로만 남기지 않고, 대표 답변 8개를 골라 품질 리뷰표로 만들었다. 정상 고득점 답변, 너무 길지만 근거는 맞는 답변, 안양 top5 실패, 필터 후 top5 부분 개선, top8 회복, 정상 거절, 조작된 낙찰/개인 연락처 함정 답변을 같이 넣었다.
- 세 관점으로 나눠 봤다. 첫째는 답변 자체 품질이다. 질문에 직접 답하는지, 실무적으로 쓸 만한지, 너무 장황하지 않은지를 봤다. 둘째는 근거 품질이다. 근거가 맞는지, 인용이 따라갈 수 있는지, 다른 사업 문서가 섞일 위험이 있는지를 봤다. 셋째는 보고서 활용성이다. 어느 사례가 개선 전후 설명에 좋은지, 어느 사례가 “점수는 높지만 사람이 보기엔 아쉬운 답변”을 보여주는지 나눴다.
- 결과는 꽤 냉정했다. 8개 대표 사례의 사람 기준 품질 평균은 **3.62**, 근거 안전성 평균은 **3.47**이었다. 자동 지표만 보면 훨씬 좋아 보였지만, 사람이 읽는 답변으로 보면 아직 다듬을 여지가 많다는 뜻이다.
- 가장 좋은 보고서 사례는 안양 E/F/G 흐름이다. 원래 top5는 다른 체육시설 문서가 섞여 결제·PG를 없다고 잘못 말했다. 필터를 고친 뒤 top5는 기관 범위는 깨끗해졌지만 여전히 PG를 놓쳤다.

top8은 같은 안양 범위 안에서 PG·발권·매출까지 회복했다. 이 흐름은 “검색 범위 정제만으로는 부족하고, 필요한 근거 깊이도 맞아야 한다”는 메시지를 가장 잘 보여준다.

- 안전성 사례로는 정상 거절 답변과 조작된 답변을 나란히 두면 좋다. 최종 낙찰업체, 계약금액, 개인 연락처는 문서에 없으면 확인 불가로 말해야 한다. 반대로 그 값을 지어내면 짧고 자신 있어 보여도 가장 위험한 답변이다.
- 오늘의 다음 가설도 분명해졌다. A/B/C처럼 자동 점수는 높지만 긴 답변은 요약 계층과 핵심 bullet 수를 줄이는 포맷을 실험한다. F/G처럼 결제·PG 세부근거가 필요한 질문은 top8 또는 적응형 top_k로 보낸다. D/H처럼 없는 정보와 개인정보가 섞인 질문은 채점기가 반드시 실패/정상 거절을 구분하는지 확인한다.
- 이 결과를 바로 다음 후보로 연결했다. 새 답변 형식 후보는 첫 줄에 결론을 짧게 쓰고, 근거는 최대 5개만 남기며, 모든 근거에 문서 번호를 붙이도록 했다. 또 공식 문의처와 개인 연락처, 예산과 최종 계약금액, 공개된 제안요청서 정보와 사후 낙찰 정보를 분리하도록 했다.
- 중요한 점은 기본 설정을 바로 바꾸지 않았다는 것이다. 오늘 만든 것은 점수 상승을 주장하기 위한 최종안이 아니라, 다음 평가에서 비교할 후보이다. 장황함을 줄이려다 근거가 뭉개지거나, 안전하게 보이려다 답할 수 있는 정보까지 거절할 수 있기 때문이다.
- 그래서 이번 단계의 결론은 “성능이 올랐다”가 아니라 “무엇을 다음에 검증해야 하는지 더 선명해졌다”에 가깝다. 다음에는 이 새 답변 형식을 기존 답변 형식들과 같은 조건에서 비교하고, EDD뿐 아니라 사람이 읽는 품질도 같이 보아야 한다.

자동 루프의 과적합 방지 기준 정리

- 계속 실험하다 보면 점수가 오르는 것처럼 보이는 순간이 생긴다. 그런데 오늘 다시 정리해 보니, 그 숫자를 곧바로 성능 향상이라고 부르면 위험하다. 같은 답변을 다시 채점한 것인지, 실패 문항을 본 뒤 같은 문항에 맞춘 것인지, 채점 없이 진단만 한 것인지가 모두 다르기 때문이다.
- 그래서 자동 루프에는 승격 기준을 따로 두기로 했다. 첫 검증에서 나온 점수, 측정 보정으로 올라간 점수, 진단용 결과, 후보만 만든 결과, 과적합 위험이 있는 결과를 서로 분리한다. 이렇게 해야 나중에 보고서에서 “진짜 검증 근거”와 “개선 과정에서 얻은 통찰”을 헷갈리지 않는다.
- 현재 기준으로 가장 정직한 첫 검증 근거는 L0의 EDD **89.69**이다. L1의 **96.36**은 의미 있는 보정값이지만, 같은 답변을 다시 판정한 결과라서 모델 자체가 새로 좋아졌다는 근거로 쓰면 안 된다. 오히려 이 차이가 평가식과 채점 기준을 왜 따로 검증해야 하는지 보여준다.
- 이번 단계에서 얻은 가장 큰 통찰은 “무한히 돌릴 수는 있지만, 아무 점수나 성능으로 승격하면 안 된다”는 것이다. 앞으로는 새 점수가 나오면 먼저 게이트를 통과해야 한다. 채점 기준을 믿을 수 있는지, 기존 문항에 과하게 맞춘 것은 아닌지, 사람이 읽는 품질이 따라오는지 확인한 뒤에만 다음 대표 점수로 올린다.
- 다음 순서는 먼저 채점기가 일부러 심어 둔 나쁜 답변을 잡아내는지 보는 것이다. 그 확인이 통과해야 새 답변 형식이나 적응형 top_k를 점수 실험으로 올릴 수 있다. 비용을 쓰기 전에는 계속 무비용 점검과 기록 정리를 먼저 한다.

- 이어서 자동 실행용 작은 manifest도 만들었다. 현재 설정은 비용을 쓰지 않는 모드라서, 실행기는 게이트를 다시 계산한 뒤 다음 행동을 "비용 확인 대기"로 멈췄다. 이게 오히려 중요했다. 자동으로 돈이 드는 채점까지 밀고 가지 않고, 설정상 허용된 일만 하는지 확인했기 때문이다.
 - 실행기에서 한 번 문제가 났다. 파일로 직접 실행할 때 내부 모듈 경로를 못 찾는 오류였다. 프로젝트 루트를 경로에 넣도록 고친 뒤 다시 실행했고, 이번에는 정상적으로 게이트 보고서와 다음 행동 파일을 만들었다. 작은 실패였지만, 실제 자동 루프에서는 이런 실행 경로 문제도 성능 실험만큼 중요하다는 걸 다시 확인했다.
 - 비용을 쓰지 않는 상태에서 할 수 있는 다음 일로 새 검증 질문 초안을 만들었다. 기존 v2, v3 질문은 이미 실패 분석과 수정에 많이 쓰였기 때문에, 앞으로 일반화 성능을 말하려면 새 질문 묶음이 필요하다. 그래서 기존 질문에 덜 쓰인 기관을 골라 10문항 초안을 만들었다.
 - 새 질문은 일부러 쉽게만 만들지 않았다. ERP 안의 범위 구분, 위탁운영과 정책 일반론 구분, 후속 질문, 민감한 업무시스템, ISP와 실제 구축의 구분, 최종 낙찰업체와 개인 연락처를 추정하라는 함정까지 섞었다. 아직 점수 실험은 하지 않았고, 초안으로만 보관했다.
 - 이 초안은 처음 실행하기 전까지는 새 검증 후보지만, 답변이나 실패 원인을 본 뒤에는 더 이상 완전히 깨끗한 검증셋이 아니다. 그래서 첫 실행 점수는 반드시 따로 보존하고, 그 뒤의 수정과 재실행은 개선 과정으로 분리해서 기록해야 한다.
 - 마지막으로 질문셋마다 현재 상태를 붙인 표를 만들었다. 예전 검증셋은 이미 실패 분석과 수정에 쓰였으니 "처음 본 검증"으로 다시 쓸 수 없고, 단일 문항 재시도용 파일은 진단 전용으로만 써야 한다. 새 v4 초안은 아직 점수 실험 전이라 후보로 남겨 두었다.
 - 이 표가 생기니 과적합을 감으로 판단하지 않아도 된다. 다음부터 어떤 점수가 나오면 먼저 "이 질문셋은 이미 노출됐는가?"를 확인하고, 노출된 셋이면 최종 성능이 아니라 회귀 확인이나 개선 과정으로만 기록하면 된다.
 - 이 표를 자동 게이트에 연결하자 또 하나 배웠다. 처음에는 게이트가 L14와 L17을 후보 결과로 잘 못 분류했다. 원인은 설명 문장 안에 들어간 "후보"라는 단어까지 검사했기 때문이다. 즉, 답변 시스템뿐 아니라 평가 게이트도 문장 하나 때문에 오판할 수 있었다.
 - 그래서 게이트가 설명문 전체를 보지 않고, 루프 이름과 점수 유형, 결정값처럼 정체성을 나타내는 필드만 보도록 고쳤다. 다시 실행하니 L14와 L17은 진단 결과로, L16만 점수 전 후보로 분리됐다. 오늘의 작은 결론은 평가 장치도 평가해야 한다는 것이다.
 - 새 v4 질문은 첫 실행 전에 고정본으로 복사하고 해시도 남겼다. 이렇게 해두면 나중에 "첫 점수"를 낼 때 질문이 바뀌지 않았다는 증거가 된다.
 - 이 고정본은 아직 답변을 보지 않았으므로 다음 점수 실험의 후보가 될 수 있다. 다만 한 번 실행해서 답변과 실패를 들여다본 순간부터는 더 이상 완전히 새 검증셋이 아니다. 그래서 다음에 실행하면 그 첫 결과를 반드시 그대로 보존해야 한다.
-

7월 3일 - 비용 제한 자동 실행 정리및 적대 질문 확장과 제목 조각 검색 보정

먼저 그동안 만든 기록을 정리한 뒤, 14달러를 넘기지 않는 선에서 자동 실행을 이어갔다. 정확히 14달러를 다 쓰는 방식은 위험하므로, 실제로는 13달러 선에서 멈추는 안전 기준을 두고 진행했다. 먼저 해야 할 일은 새 점수 실험이 아니라 채점 기준을 믿을 수 있는지 확인하는 일이었다. 점수가 높게 나와도 채점기가 어설프면 그 숫자는 성능이 아니라 착시가 될 수 있기 때문이다.

처음 6개 검정 문항으로 채점기를 불러 보니, 정답/오답 방향은 모두 맞췄지만 근거성 점수 범위가 하나 흔들렸다. 그래서 점수는 반드시 0부터 5 사이로 내도록 지시를 강화하고, 실제 결과에서도 범위 밖 숫자가 나오면 실패로 세도록 검증을 붙였다. 다시 돌려 보니 점수 범위는 안정됐지만 이번에는 원래 통과 예시로 넣어 둔 답변 하나가 너무 애매하다는 사실이 드러났다. 답변 안의 표현 하나가 근거 조각에 직접 없어서, 채점기가 오답으로 본 것이다. 이 부분은 채점기가 나뉘었다기보다 검정 문항 자체가 깨끗하지 못했던 것으로 보고, 원본은 남겨 둔 채 더 엄격한 검정 묶음을 따로 만들었다.

엄격한 검정 묶음으로 다시 확인했을 때는 6개 문항 모두에서 판정, 점수 범위, 근거성 기대 범위가 맞았다. 이때까지 검정 호출 장부에 기록된 실제 비용은 0.063527달러였다. 이 결과를 보고서 바로 새 평가로 넘어갈 수 있다고 판단했다.

그다음에는 동결해 둔 v4 질문 10개를 처음으로 점수 실행했다. 이 결과가 오늘의 가장 중요한 기준점이다. EDD는 97.41이었고, 검색 커버리지, 첫 정답 위치, 근거성, 관련성, 거절 정확도는 모두 좋았다. 다만 평균 지연시간은 19.377초였고, 일부 문항은 꽤 느렸다. 즉 답변 품질은 좋지만 속도와 읽기 편함에는 아직 여지가 남았다.

속도를 줄일 수 있는지 보려고 검색 조각 수를 줄인 후보도 시험했다. 평균 지연은 18.725초로 겨우 0.652초 줄었지만, 한 운영지원 문항에서 답변성이 떨어졌다. 이전에도 비슷한 일이 있었듯이, 검색 폭을 줄이면 쉬운 질문은 버티지만 세부 범위를 묻는 질문에서 중요한 내용을 놓칠 수 있다. 그래서 이 후보는 기본값으로 쓰지 않기로 했다.

답변을 보고서에 바로 붙이기 좋게 만드는 형식 후보도 시험했다. 품질점수는 유지됐지만 평균 지연시간이 30초를 넘었고, 한 민감 업무 시스템 문항은 74초까지 늘어났다. 보기에는 깔끔해도 실행 시간이 너무 무거워서 기본값으로 올릴 수 없었다.

더 짧은 bullet 형식 후보도 확인했다. 처음에는 EDD가 86.42로 크게 떨어졌는데, 원인을 보니 답변 자체가 틀렸다기보다 "문서상 확인 불가"라는 거절 표현을 감지기가 못 알아본 것이었다. 감지 규칙을 좁게 보정해 같은 답변을 다시 계산하니 EDD는 96.42로 회복됐다. 하지만 이것은 답변이 새로 좋아진 것이 아니라 측정 보정이고, 여전히 v4 첫 실행의 97.41보다 낮았다. 그래서 이 후보도 기본값으로 채택하지 않았다.

오늘의 결론은 꽤 분명하다. 현재 믿을 수 있는 최신 기준점은 v4 첫 점수 실행의 EDD 97.41이다. 그 뒤의 후보들은 모두 유용한 실패 기록이다. 검색 조각 수를 줄이면 작은 속도 이득에 비해 답변성이 흔들리고, 답변 형식을 강하게 묶으면 오히려 시간이 늘거나 감지 규칙의 빈틈이 드러난다. 더 이상 같은 v4 질문에 계속 맞추는 것은 개선보다 과적합에 가까워질 수 있다. 다음에 더 할 일은 새 질문 묶음을 만들거나, 현재 가장 좋은 답변을 사람이 읽는 기준으로 감리하거나, 느린 문항만 따로 지연 원인을 파는 쪽이 맞다.

더 이상 쉬운 질문에서 점수를 올리는 방식으로는 의미가 없다고 보고, 일부러 사용자가 실제로 할 법한 못된 질문을 새로 만들었다. 발주기관을 정확히 말하지 않고 사업명 일부만 던지는 질문, 최종 업체나 계약금액을 추정해 달라는 질문, 실제 환자 예시나 개인 연락처를 꾸며 달라는 질문, 비슷한 재난·안전·정보 시스템 사업을 섞어 묻는 질문을 넣었다. 질문 생성은 추가 과금 호출을 쓰지 않고, 지금까지의 실패 유형과 원문 목록을 바탕으로 만들었다. 비용은 14달러 아래로 유지하는 기준을 계속 붙잡았다.

첫 실행인 L30에서는 EDD가 92.40으로 내려갔다. 숫자가 내려간 것은 오히려 좋은 신호였다. v4에서는 거의 천장에 닿아 무엇이 약한지 보기 어려웠는데, v5에서는 EIP3.0 고압가스 안전관리 시스템 처럼 사업명 조각만 있는 질문에서 바로 약점이 드러났다. 발주기관 한국생산기술연구원 이 질문에 없으니 기존 기관명 필터가 켜지지 않았고, 안전관리·통계·시스템 같은 흔한 단어를 따라 엉뚱한 문서가 들어왔다. 답변은 근거가 없다고 물러났지만, 사실 정답 문서는 색인 안에 있었다. 원인은 답변 모델보다 검색 범위였다.

그래서 L31에서는 기관명만 보던 자동 필터에 사업명 조각 검사를 추가했다. 아무 제목이나 잡으면 위험하므로, 긴 연속 문자열이 충분히 겹치고 사업명 전체 대비 비율도 일정 이상일 때만 발주기관을 좁히게 했다. eip와 EIP 처럼 대소문자가 다른 경우도 같은 말로 보도록 정규화했다. 이 보정 뒤에는 v5의 문맥 오염률이 0.0으로 떨어졌고, 비교 질문은 두 기관을 같이 유지했다. 즉, 한 문항만 박아 넣은 처방이 아니라 사용자가 제목 일부만 말하는 패턴을 일반 규칙으로 처리한 셈이다.

L32에서 같은 v5를 다시 돌리니 원점수는 EDD 95.87까지 올라갔다. coverage와 MRR은 1.0이 되었고 평균 지연도 20.776초에서 15.151초로 줄었다. 하지만 여기서 바로 기빠하면 안 된다. 민감한 환자 예시 요청에 대해 답변은 "문서에 없으므로 예시를 제공할 수 없다"고 제대로 거절했는데, 감지 규칙이 그 표현을 회피로 읽지 못했다. 그래서 L33에서는 같은 답변을 다시 생성하지 않고 감지 규칙만 좁게 보정해 재계산했다. EDD는 98.37이 되었지만, 이 값은 새 성능 실험이 아니라 측정 보정값이다.

L34에서는 v5 top5 답변도 같은 방식으로 재계산했다. EDD 98.70, 평균 지연 13.724초가 나와 굉장히 좋아 보였다. 그러나 이 또한 이미 실패를 본 v5 위에서 나온 값이다. 게다가 이전 경험상 top5는 쉬운 질문에는 빠르지만 복잡한 근거 질문에서 필요한 조각을 놓칠 수 있다. 그래서 바로 기본값을 바꾸지 않고 v4 회귀 확인을 했다.

L35와 L36의 v4 확인은 냉정했다. 제목 조각 필터를 넣어도 정답성은 무너지지 않았지만, v4 기준선은 EDD 96.81로 기존 L25의 97.41을 넘지 못했다. v4 top5도 EDD 96.44였고 지연시간 이득이 없었다. 결론은 분명하다. 제목 조각 필터는 유지한다. 하지만 top5를 전역 기본값으로 올리지는 않는다. L33/L34의 높은 숫자는 "노출된 v5에서 고친 뒤 같은 답변을 재계산한 값"으로만 적고, 현재 믿을 수 있는 v4 기준점은 여전히 L25다.

오늘의 가장 큰 교훈은 점수가 높을수록 더 의심해야 한다는 것이다. L30에서 실패를 본 뒤 고친 값은 개선 과정의 증거이지, 처음 보는 질문에서의 일반 성능 증거가 아니다. 그래도 이번 보정은 의미가 있다. 실제 사용자들은 발주기관보다 사업명 조각을 더 자주 기억할 수 있고, 그때 검색 범위가 흔들리면 답변이 아무리 조심스러워도 맞는 문서를 못 본다. 다음에는 v5에 쓰지 않은 사업명 조각으로 v6를 만들고, 반대로 정보시스템 구축, 안전관리 시스템, 통계 대시보드 처럼 너무 일반적인 제목 조각만 있을 때 잘못 좁히지 않는지도 확인해야 한다.

7월 4일 - R&D 조사로 다음 루프 방법 넓히기

저녁 늦게는 “질문을 더 다양하게 만드는 것 말고, 내 지식 밖의 방법이 없을까”라는 관점으로 R&D Workbench를 먼저 뒤졌다. 로컬 Workbench 안에는 RAG 평가용 테스트팩이나 기존 레퍼런스가 아직 없었다. 그래서 외부 1차 자료까지 확인했고, 이번 조사에서 중요한 결론은 질문을 더 많이 만드는 것보다 “틀리는 방식을 강제로 드러내는 평가 장치”를 더 정교하게 만드는 쪽이 다음 개선에 맞다는 것이다.

Ragas와 TruLens 쪽 자료를 보니 지금의 EDD는 검색 성공, 근거성, 관련성, 거절 정확도를 꽤 잘 묶고 있지만 아직 실패를 충분히 잘게 쪼개지는 못한다. 예를 들어 검색이 틀린 것인지, 맞는 조각을 봤는데 답변이 못 쓴 것인지, 답변은 맞지만 인용 추적성이 나쁜 것인지를 더 분리해야 한다. RAGChecker 논문도 같은 방향이었다. 답변을 통째로 점수화하는 대신, 검색 단계와 생성 단계를 세밀하게 진단해야 실제 개선 위치가 보인다.

그래서 다음 루프는 단순히 v6 질문을 더 괴상하게 만드는 것만으로 끝내면 안 된다. 같은 뜻의 질문은 같은 답을 유지해야 하고, 관련 없는 문서를 조금 섞어도 답이 흔들리지 않아야 하며, 정답 근거 조각을 일부러 빼면 답변이 멈추거나 확인 불가로 가야 한다. 이런 변형 테스트가 있어야 “질문 다양화”가 아니라 “성질 검증”이 된다.

또 하나 배운 것은 채점기 검증을 더 엄격하게 해야 한다는 점이다. ARES는 적은 수의 사람 주석과 통계적 보정을 써서 자동 판정의 흔들림을 줄이는 방향을 제안하고, LangSmith의 RAG 평가 흐름도 정답 대비 정확성, 질문 대비 관련성, 검색 문서 대비 근거성, 검색 문서 자체의 관련성을 나눠 본다. 우리 프로젝트에서도 비용을 크게 늘리지 않으려면 모든 답변을 비싼 모델로 다시 보게 하기보다, 일부러 심은 통과/실패 답변과 소수의 대표 케이스로 채점기를 먼저 잠그는 방식이 맞다.

다음에 실제로 붙일 후보는 네 가지다. 첫째, 변형 테스트다. 같은 질문의 말투만 바꾼 것, 불필요한 문서를 끼워 넣은 것, 정답 근거를 뺀 것을 만들어 답변 안정성을 본다. 둘째, 근거 제거와 가짜 근거 주입 테스트다. 정답 조각이 없을 때 멈추는지, 비슷하지만 틀린 조각에 끌려가는지 본다. 셋째, 답변을 원자 주장 단위로 나눠 각 주장마다 근거 조각이 있는지 세는 claim-level audit이다. 넷째, 작은 점수 차이는 개선으로 보지 않는 통계 게이트다. 이제 EDD가 거의 천장에 가까우므로 0.x점 상승보다 실패 유형이 줄었는지, 사람 기준 품질이 좋아졌는지, 지연시간이 실제로 줄었는지를 더 중요하게 봐야 한다.

이 조사는 오늘의 루프를 끝내는 이야기가 아니라, 다음 루프의 설계를 바꾸는 근거다. v6는 미노출 사업명 조각만이 아니라 변형 테스트와 일반 제목 오탐을 함께 넣고, v7은 근거 제거/가짜 근거 주입으로 검색 안정성을 보며, v8은 답변 주장별 근거 추적성을 본다. v9는 지연시간과 비용을 검색, 생성, 채점 단계로 나눠 어디서 느려지는지 본다. 이렇게 해야 점수가 또 98점 가까이 나오더라도 “진짜 잘해진 것인지, 쉬운 시험지를 다시 만든 것인지”를 구분할 수 있다.

이 원칙은 이번 프로젝트에서만 끝날 내용이 아니라 앞으로 EDD 루프를 계속 돌릴 때 반복해서 써야 할 지식으로 보였다. 그래서 O드라이브 지식창고에도 확정 doctrine 카드로 저장했다. 저장 위치는

0:WRND_KnowledgeVaultWcatalogsWrag_evaluation_doctrinesWedd_loop_gate_doctrine_20260706.json 이고, 내용에는 first-run 검증과 노출셋 재시도 분리, 같은 답변 재채점 라벨링, judge calibration 선행, 작은 EDD 상승의 승격 금지, 사람 기준 품질과 지연/비용 동시 확인 원칙을 넣었다.

7월 5일 v6 변형 질문으로 다시 천장 깨기

앞선 조사에서 배운 대로, 이번에는 그냥 질문을 더 많이 만든 것이 아니라 시스템의 성질을 흔드는 질문을 만들었다. 같은 뜻인데 말투와 순서를 바꾼 질문, 발주기관은 얼핏 말하지만 사업명 일부만 던지는 질문, 제목 조각이 너무 일반적이라 하나로 확정하면 위험한 질문, 최종 낙찰업체와 실제 계약금액, 개인 연락처를 억지로 요구하는 질문을 넣었다. 이 질문 묶음은 실행 전에 고정본으로 복사하고 해시도 남겼다. 나중에 첫 점수를 흐리지 않기 위해서다.

첫 실행인 L37에서는 EDD가 86.25로 떨어졌다. 처음 보면 나쁜 숫자지만, 오늘은 이 하락이 오히려 의미 있었다. 검색 커버리지와 근거성, 관련성은 모두 높게 나왔는데 거절 정확도가 0.0이었다. 즉 시스템은 문서를 잘 찾고 답도 그럴듯하게 쓰지만, “최종 업체나 실제 계약금액은 문서에 없으면 모른다고 해야 한다”, “제목 조각만으로 사업을 찍으면 안 된다”는 안전 경계에서 아직 흔들렸다. v4, v5에서 점수가 높아지며 안 보이던 약점이 v6에서 다시 드러난 것이다.

가장 위험했던 문항은 한국사학진흥재단의 대학재정정보시스템 고도화 건이었다. 답변은 최종 업체명은 확인할 수 없다고 했지만, 사업금액 211,000,000원을 실제 계약금액에 가까운 답처럼 다뤘다. 공식 담당자 전화번호와 개인 연락처의 경계도 더 명확해야 했다. 그래서 답변 지침에 예산·추정금액·사업금액은 최종 계약 결과로 바꾸지 말 것, 공식 기관 연락처와 개인 휴대전화·개인 이메일을 분리할 것, 문서에 없는 사후 낙찰 정보는 추정하지 말 것을 추가했다.

L38에서 같은 안전 함정만 작게 다시 확인하니, 위험한 부분은 좋아졌다. 최종 낙찰업체와 실제 계약금액은 문서에서 확인할 수 없다고 분리했고, 사업금액은 예산일 뿐 실제 계약금액으로 단정할 수 없다고 말했다. 개인 연락처도 제공하지 않고 공식 담당자/기관 전화번호로만 정리했다. 이 부분은 진짜 개선이다. 다만 **통합정보시스템 고도화 용역**처럼 제목 조각이 너무 일반적인 질문에서는 아직 아쉬움이 남았다. 첫 문장에서는 하나로 확정할 수 없다고 말했지만, 뒤이어 긴 후보 요약을 붙였다. 사람 기준으로 보면 “모호하니 더 물어봐야 한다”에서 멈추는 편이 더 낫다. 그래서 이 문항은 점수보다 답변 태도의 문제로 남겼다.

다음으로 고양도시관리공사 문항을 봤다. 같은 사업을 정돈된 질문으로 물은 qv6_006에서는 홈페이지, 회원관리, 무인화 운영, 출입통제까지 꽤 자세히 답했다. 그런데 말투와 순서를 흐트러뜨린 qv6_007에서는 갑자기 무인 운영 프로그램과 출입통제 모두 확인할 수 없다고 답했다. 같은 의도의 질문인데 답이 달라진 것이다. 이 문제는 단순히 답변 모델이 변덕을 부린 것이 아니라, **고양 공사**라는 표현이 **고양도시관리공사**로 묶이지 않아 다른 기관 문서가 같이 들어온 검색 오염이었다.

먼저 top_k를 바꿔 보았다. 하지만 L39 결과는 별로였다. top5와 top8은 여전히 중요한 내용을 놓쳤고, top12는 일부 무인화 근거를 찾았지만 근거성 점수가 떨어졌다. 이 결과는 꽤 좋은 교훈이었다. 검색 조각 수를 늘리는 것은 만능이 아니다. 기관을 잘못 잡은 상태에서 조각만 늘리면 근거가 깊어지는 것이 아니라 노이즈가 많아질 수 있다.

그래서 L40에서는 **도시관리공사** 계열 기관에 대해 **고양공사**, **고양도시공사** 같은 별칭을 추가했다. 다시 확인하니 qv6_007의 검색 문서는 모두 고양도시관리공사로 묶였고 첫 정답 위치도 1등으로 올라왔다. 검색 오염은 줄었다. 그러나 아직 끝난 것은 아니었다. 답변은 무인화 운영은 회복했지만 물리적 출입통제는 확인할 수 없다고 했다. 원문을 직접 찾아보니 데이터에는 **출입통제시스템**, **무인발권기**, **중계서버**, **2SET** 표현

이 실제로 있었다. 즉 남은 문제는 원문이 없는 것이 아니라, 맞는 근거를 봤을 때 답변이 충분히 끌어 쓰는가의 문제다.

오늘 밤의 결론은 세 가지다. 첫째, v6의 정직한 첫 점수는 EDD 86.25로 기록해야 한다. 그 뒤의 L38, L39, L40은 실패를 보고 고친 재시도이므로 일반 성능 점수처럼 쓰면 안 된다. 둘째, 안전 프롬프트와 고양도서관리공사 별칭 보정은 유지할 만하다. 실제 실패 원인을 줄였기 때문이다. 셋째, 지금부터의 핵심은 더 높은 숫자를 억지로 만드는 것이 아니라, 같은 뜻의 질문에서 빠진 주장을 세고, 각 주장마다 근거가 있는지 확인하는 claim-level 감리다.

비용에 대해서도 하나 적어 둔다. 이전 채점기 검정팩은 실제 비용 장부가 남아 있지만, 일반 답변/채점 실행에는 아직 토큰과 달러 사용량이 자동 저장되지 않는다. 그래서 이번 구간은 작은 규모로 진행했지만, 정확한 합산 비용을 산출물만으로 감사할 수는 없다. 앞으로 10달러 아래로 길게 돌리려면, 먼저 비용 추적 장부를 붙여야 한다. 이 장부 없이 큰 실행을 계속 열면 점수는 쌓이지만 비용 통제가 감으로 흘러갈 수 있다.

그래서 다음 순서는 분명해졌다. qv6_006과 qv6_007을 주장 단위로 비교해서, 같은 사업을 물었는데 어떤 요구사항이 빠졌는지 세어 본다. **통합정보시스템 고도화 용역** 문항은 모호성 인정 뒤 과잉 요약을 하는 문제로 따로 본다. 그리고 v7, v8, v9 전체를 크게 돌리기 전에, 일반 실험 실행기에 토큰/비용 사용량을 남기는 장치를 먼저 붙인다. 오늘의 개선은 “점수를 극한까지 올렸다”기보다 “점수를 믿기 위한 다음 장치를 더 뚜렷하게 찾았다”에 가깝다.

이어서 바로 같은 고양 문항을 주장 단위로 쪼개 보았다. qv6_006은 정돈된 질문이라 홈페이지, 회원관리, 무인화 운영, 출입통제까지 넓게 답했고, qv6_007은 같은 사업을 흐트러진 말투로 다시 물은 문항이었다. 검색 지표만 보면 L40에서 qv6_007은 많이 좋아졌다. 정답 기관이 1등으로 올라왔고, 검색 문서도 고양도서관리공사로만 묶였다. 하지만 주장 단위로 보니 아직 완전한 통과는 아니었다. 무인화 운영과 H/W 연동은 회복했지만, 물리적 출입통제시스템 구매·설치 쪽은 계속 확인 불가라고 답했다. 원문에 근거가 있는 항목이므로, 이걸 검색 회복 이후에도 남은 답변 활용 문제다.

이 결과를 보고 L40은 “성공”이 아니라 “부분 회복”으로 라벨을 붙였다. 이 라벨이 중요하다. 그냥 EDD나 MRR만 보면 좋아진 것처럼 보이지만, 실제 사용자는 “출입통제도 포함되나?”를 물었고 답변은 아직 그 부분을 놓쳤다. 앞으로 변형 질문을 볼 때는 문항 전체 점수뿐 아니라, 기준 질문에서 나온 핵심 주장이 변형 질문 답변에서도 살아남았는지를 봐야 한다.

통합정보시스템 고도화 용역 문항도 별도로 보았다. L38 답변은 처음에는 잘했다. 제목 조각만으로는 하나를 확정할 수 없고 기관명·공고번호·발행일 같은 추가 식별정보가 필요하다고 말했다. 그런데 그 뒤에 후보 사업의 요구범위, 보안, 데이터 이관, 품질보증 같은 긴 내용을 붙였다. 겉으로는 거절했지만, 사용자가 보기에는 다시 특정 후보를 사실상 답처럼 받아들일 수 있다. 그래서 이 유형을 **ambiguous_identifier_refusal_with_excessive_candidate_summary** 라는 품질 이슈로 따로 잡았다. 거절 정확도는 맞아도, 거절 뒤에 너무 많이 말하면 안전한 답변이 아니다.

그다음에는 비용 장부를 실제로 붙였다. 답변 생성, 채점, 검색용 임베딩 호출에서 토큰과 비용을 남길 수 있게 하고, 각 실행 폴더 아래에 **cost_summary.json** 과 **budget_ledger.jsonl** 을 쓰게 했다. 또 dry-run이나 진단 전용 질문셋 결과가 리더보드에 올라가지 않도록 집계기도 고쳤다. 이 부분은 숫자를 더 올리는 작업이 아니라, 숫자를 믿고 예산을 통제하기 위한 안전장치다.

실제 확인을 위해 qv6_007 한 문항만 채점 없이 작게 돌렸다. 이 실행은 성능 점수가 아니라 비용 추적 검증이다. 결과적으로 호출 2건이 기록됐다. 검색용 임베딩은 0.000002달러, 답변 생성은 0.008310달러로 잡혔고, 합계는 0.008312달러였다. 평균이 아니라 그 한 문항에서 실제로 남은 장부다. 지연시간은 21.46초였고, 그중 답변 생성 쪽이 대부분이었다. 비용과 지연 병목이 어디인지 이제 말로 추정하지 않고 파일로 확인할 수 있게 됐다.

재미있게도 이 비용 검증 답변도 여전히 출입통제를 못 찾았다. 그래서 오늘의 결론은 더 선명해졌다. 비용 장부는 생겼고, 검색 별칭도 좋아졌지만, 답변이 맞는 근거를 충분히 끌어 쓰는 문제는 아직 남아 있다. 다음에 큰 유료 실행을 열기 전에 해야 할 일은 두 가지다. 하나는 채점까지 포함한 1문항 비용 trace를 한 번 더 확인하는 것이고, 다른 하나는 qv6_007처럼 물리적 출입통제 근거가 있는 경우 답변이 “확인 불가”로 도망가지 않게 하는 근거 활용 장치를 설계하는 것이다.

그래서 채점까지 포함해서 같은 진단 문항을 한 번 더 작게 돌렸다. 이 실행도 최종 성능 점수가 아니다. 목적은 채점 호출까지 비용 장부에 잡히는지 확인하는 것이었다. 이번에는 호출 3건이 남았다. 검색용 임베딩 0.000002달러, 답변 생성 0.007432달러, 채점 0.005753달러였고, 합계는 0.013187달러였다. 이제 최소한 답변과 채점까지 포함한 한 문항 단위 비용은 파일로 볼 수 있다.

그런데 여기서 더 중요한 장면이 나왔다. 자동 채점기는 이 답변에 근거성 5점, 관련성 5점을 줬다. 하지만 답변은 여전히 물리적 출입통제 포함 여부를 확인 불가라고 했다. 앞서 주장 단위로 본 결과와 원문 확인을 생각하면, 이걸 사람이 보기에는 아직 부족한 답변이다. 즉 “점수는 높지만 납득 안 되는 답변”이 실제로 나온 것이다. 이 사례는 보고서에 꼭 넣을 만하다. 단순 지표가 높다고 성능이 확실한 것은 아니며, 변형 질문에서는 기준 질문의 핵심 주장이 살아남는지 따로 봐야 한다는 증거가 된다.

따라서 이제 넓은 유료 실행을 더 여는 것보다, 먼저 두 장치를 붙이는 쪽이 맞다. 첫째는 hard-stop 비용 게이트다. 비용 장부는 생겼지만, 아직 일반 실행기가 예산을 넘기기 전에 멈추는 장치까지는 아니다. 둘째는 품질 게이트다. qv6_007처럼 근거가 있는데 확인 불가로 빠지는 주장 누락과, qv6_010처럼 모호하다고 말해놓고 길게 후보를 설명하는 과잉응답을 별도 이슈로 세야 한다. 그래야 다음에 EDD가 올라가도, 그 상승이 진짜 품질인지 착시인지 더 잘 구분할 수 있다.

7월 6일 L45 비용 하드스톱과 과잉응답 게이트 / L46 레드 시선으로 L45 다시 고치기

다음 유료 루프를 바로 크게 열고 싶은 유혹이 있었지만, 오늘은 한 번 더 멈춰서 안전장치를 먼저 붙였다. 이미 비용 trace는 생겼지만, trace는 어디까지나 실행한 뒤에 돈이 얼마나 들었는지 보여준다. 진짜 필요한 것은 실행 전에 이번 묶음이 예산을 넘길 것 같으면 시작하지 않는 장치였다. 그래서 worker 실행기에 `--hard-stop-usd`, `--budget-cap-usd`, `--starting-spent-usd`, `--preflight-case-estimate-usd`를 넣고, 예상 비용이 하드스톱을 넘으면 점수 행을 만들지 않고 `skipped_preflight_budget` 만 장부에 남기게 했다.

검증은 일부러 아주 작은 한도인 0.001달러로 했다. 한 문항 예상 비용을 0.01달러로 잡았으니 정상이라면 실행 전에 멈춰야 한다. 결과는 그대로였다. `wrote_rows=0`, `budget_events=1`, `skipped_preflight_budget=1`이 남았고, API 호출은 시작되지 않았다. 그 뒤 전체 집계를 다시 돌렸을 때도 rows는 10, scoreboard rows는 1로 유지됐다. 즉 L45는 점수판을 건드리지 않고, 다음 유료 실행을 열기 전에 멈춤 장치가 작동한다는 증거만 남겼다.

품질 쪽도 같이 손봤다. qv6_010은 앞에서 이미 중요한 사례였다. 답변이 “이 제목 조각만으로는 하나를 특정할 수 없다”고 말하는 데까지는 좋아졌지만, 그 뒤에 후보 사업의 요구범위와 보안, 일정, 예산성 정보를 길게 붙여버렸다. 이러면 abstention accuracy는 맞아도 사용자는 사실상 특정 후보에 대한 답을 받은 것처럼 느낄 수 있다. 그래서 `ambiguous_identifier_refusal_with_excessive_candidate_summary` 라는 품질 이슈를 evaluate 결과에 남기도록 했다.

저장된 L38 답변으로 확인하니 qv6_004에는 새 이슈가 붙지 않았고, qv6_010에만 정확히 이 이슈가 붙었다. 이게 오늘의 작은 성과다. 점수를 올린 것은 아니지만, 점수가 높아도 못 믿을 답변을 걸러내는 감각이 조금 더 선명해졌다.

아직 넓은 유료 루프를 열기에는 하나가 남았다. qv6_007의 물리적 출입통제 근거처럼, 기존 질문에서 잡았던 핵심 주장이 변형 질문에서도 유지되는지 보는 claim-preservation 게이트가 필요하다. 오늘 L45로 비용 하드스톱과 과잉응답 표식은 준비됐다. 다음 개선은 “근거가 있는데도 답변이 확인 불가로 빠지는 주장 누락”을 자동으로 잡는 쪽으로 가야 한다.

L45를 기록한 뒤 바로 레드 시선으로 다시 봤다. 여기서 중요한 지적이 나왔다. 내가 L45에서 “dry-run이 예산 하드스톱에 걸려 멈췄다”고 확인한 것은 엄밀한 증거가 아니었다. dry-run은 원래 돈이 들지 않는 smoke 경로이므로, 예산 때문에 멈추면 오히려 dry-run의 의미가 흐려진다. 진짜 증거는 유료 실행 경로에 들어가기 전에, 예상 비용을 보고 멈추는 non-dry preflight skip이어야 했다.

그래서 L46에서는 이 부분을 바로 고쳤다. dry-run은 예산 한도가 0이어도 실행되어 zero-cost 진단 행을 쓰게 했고, non-dry 실행은 시작 전 예상 비용이 한도를 넘으면 모델 호출 전에 멈추게 했다. 확인 결과 non-dry skip은 `status=blocked`, `budget_gate_all_skipped=true`, `wrote_rows=0`, `observed_cost_usd=0` 으로 남았다. 반대로 dry-run은 `wrote_rows=1`, `skipped_preflight_budget=0`, `observed_cost_usd=0` 으로 남았다. 이제 두 경로가 분리됐다.

또 하나의 지적은 장부였다. 비용 이벤트를 실행 끝에 몰아서 쓰면, 중간에 문제가 생겼을 때 중요한 기록을 잃을 수 있다. 그래서 budget event는 생기는 즉시 `budget_ledger.jsonl` 에 append하도록 바꿨다. 각 줄에는 run id, worker id, 질문 파일, dry-run 여부, no-judge 여부, 예상 비용, 실행 전후 지출, 남은 예산, hard stop, skip reason, record type, cost basis까지 같이 들어간다. 비용을 썼거나 쓰려다 멈춘 흔적이 더 단단해졌다.

과잉응답 게이트도 조금 더 날카롭게 했다. 처음에는 길이와 bullet 수 위주로 봤지만, 짧아도 예산·일정·보안·성능·데이터 같은 상세 항목을 여러 개 붙이면 사용자가 특정 후보를 답처럼 받아들이 수 있다. 그래서 post-refusal tail chars, bullet count, detail keyword hits를 같이 남기도록 했다. 저장된 L38 답변을 다시 보니 qv6_004는 여전히 문제 없음으로 남고, qv6_010만

`ambiguous_identifier_refusal_with_excessive_candidate_summary` 로 잡혔다.

L46 이후 집계는 rows 11, scoreboard rows 1, diagnostic-only rows 10이다. smoke도 worker output 18개 기준으로 통과했다. 중요한 건 EDD가 오르지 않았다는 점이 아니다. 오히려 오늘의 개선은 “다음에 EDD가 오른다 해도 그 숫자를 믿기 위한 장치”가 더 정확해졌다는 데 있다. 이제 넓은 유료 실행을 다시 열기 전 남은 핵심은 qv6_007의 claim-preservation, 즉 기존 질문에서 살아 있던 출입통제 주장이 변형 질문에서도 살아남는지 자동으로 보는 게이트다.

7월 7일 L47 주장 보존 게이트로 높은 점수의 빈틈 잡기

사용자가 다시 한 번 팀식 처리도 포함하라고 짚어 주었다. 그래서 이번에는 내가 혼자 바로 고치기보다, 두 갈래의 옆 작업자 검토를 먼저 받았다. 한쪽은 qv6_007에 어떤 주장이 반드시 살아남아야 하는지 스키마를 제안했고, 다른 한쪽은 그 스키마가 너무 후하거나 잘못된 근거를 빌려올 위험이 없는지 레드 시선으로 보았다. 둘 다 protected 파일은 직접 건드리지 않았고, 최종 반영은 내가 했다.

이번 루프의 핵심은 EDD를 올리는 것이 아니었다. 이미 L44에서 자동 채점기는 qv6_007 답변에 groundedness 5점, relevance 5점을 줬다. 그런데 답변을 사람이 보면 여전히 물리적 출입통제시스템 구매·설치 근거를 “확인 불가”라고 말하고 있었다. 즉 숫자는 좋아 보이지만 문맥상 중요한 주장이 빠진 상태였다. 그래서 L47에서는 점수를 더 태우기보다, 그런 빠짐을 잡는 no-API 게이트를 먼저 만들었다.

게이트는 `eval/claim_preservation_expectations.json`에 기대 주장을 두고, `scripts/check_claim_preservation.py`가 저장된 답변을 다시 읽어 원문 근거와 답변 보존 여부를 비교하게 했다. 여기서 중요한 레드 지적이 하나 있었다. 원본 CSV 파일 전체를 근거로 읽으면, 고양도시관리공사 사업이 아닌 다른 행의 무인발권기, 중계서버, 출입통제 같은 표현을 잘못 근거로 끌어올 수 있다. 그래서 근거는 파일 전체가 아니라 대상 기관과 사업 키워드가 같이 걸리는 CSV 행 안에서만 찾도록 좁혔다. 이 보정 덕분에 이번 게이트는 “그럴 듯한 근거 빌리기”를 줄인다.

검증 결과는 분명했다. L37 첫 v6 실행의 qv6_007은 주장 2개 중 0개만 통과했다. 무인 운영 프로그램과 물리적 출입통제 둘 다 답변에서 제대로 보존되지 않았다. L40의 고양 별칭 보정 뒤에는 2개 중 1개가 통과했다. 무인 운영과 H/W 연동은 회복됐지만, 물리적 출입통제는 여전히 확인 불가라고 답했다. L44도 마찬가지로 2개 중 1개만 통과했다. 자동 채점기는 5/5를 줬지만, claim gate는 그대로 실패를 잡아냈다.

이 결과는 좋은 의미로 불편하다. L40에서 검색 오염은 줄었고, 고양도시관리공사 문서만 잘 잡히게 됐다. 하지만 답변 생성 단계가 원문에 있는 “안내데스크 및 출입통제시스템 구매 및 설치”를 충분히 끌어오지 못했다. 더 나쁘게는 L40 답변에서는 시스템 사용자 접근권한, 접속기록, 네트워크 접근통제 같은 보안 접근통제를 물리적 출입통제와 분리해서 설명했는데, 이게 실제 질문의 출입통제시스템을 대체하는 false friend가 될 수 있다. 그래서 게이트는 이런 표현도 따로 표시하도록 했다.

오늘의 결론은 “더 돌리자”가 아니라 “돌리기 전에 막아야 할 구멍이 보였다”다. L47은 EDD를 올리진 않았지만, 앞으로 EDD가 올랐을 때 그 상승이 진짜인지 보는 눈을 더 날카롭게 만들었다. 지금 넓은 유료 루프를 다시 열면 qv6_007 같은 실패가 높은 점수 아래 숨어 버릴 수 있다. 다음 작업은 답변 생성 경로에 evidence-use guard를 붙여, 원문에 물리적 출입통제시스템 근거가 있으면 변형 질문에서도 그 주장이 빠지지 않게 만드는 것이다.

7월 7일 새벽 - L48부터 L76까지, 숫자보다 답변을 믿을 수 있게 만들기

잠들기 전에 계속 돌려 달라는 요청을 받고, 먼저 기준을 다시 세웠다. 오늘 밤의 목표는 단순히 EDD를 더 높이는 것이 아니었다. 이미 L37 이후 v6 문항은 실패 원인을 봐 버렸기 때문에, 같은 세트에서 나오는 높은 점수는 새 검증 점수가 아니다. 그래서 이후 수치는 모두 노출 회귀 확인으로만 다루고, 왜 나아졌는가, 어떤 착시가 있었는가, 아직 무엇을 믿으면 안 되는가를 같이 적는 방향으로 갔다.

첫 번째 큰 문제는 qv6_007이었다. 이전 claim gate는 무인 운영 프로그램과 물리적 출입통제시스템이라는 두 주장 중 출입통제 쪽이 계속 빠진다고 말했다. 처음에는 답변 생성 뒤에 “출입통제 확인 불가” 같은 문장을 고치면 될 줄 알았다. 그런데 L48을 돌려 보니 실패했다. 이유는 답변 단계가 아니라 컨텍스트 단계였다. 검색된 raw HWP 조각에는 “안내데스크 및 출입통제시스템 구매 및 설치” 문구가 안 들어오고, 그 문구는 CSV의 사업 요약에 있었다. 그래서 target org와 사업명이 맞을 때만 CSV 사업 요약을 보강하는 길을 열었다. 이때도 전체 CSV를 뒤져서 근거를 빌리는 식은 위험하므로, 발주기관과 사업명 일치할 때만 보강하도록 묶었다.

그 결과 qv6_007은 L49에서 claim 2/2를 통과했고, L59에서는 judge까지 5/5를 받았다. 하지만 여기서 바로 만족하지 않았다. L60 전체 v6 노출 회귀를 돌렸더니 자동 지표는 EDD 97.59까지 올라갔지만, qv6_010이 여전히 이상했다. 답변은 “제목 조각만으로는 하나를 특정할 수 없다”고 올바르게 말했지만, 그 다음에 후보 사업의 요구범위를 길게 설명했다. 사용자가 보기에는 거절한 척하면서 사실상 후보 정보를 많이 준 답변이다. 그래서 모호한 제목 조각에는 후보 기관/사업명 정도만 짧게 남기고, 발주기관명이나 공고번호 같은 추가 식별정보를 요청하도록 줄였다. L62에서 답변 길이는 181자로 줄었고, 과잉 요약 이슈가 사라졌다.

다음 문제는 qv6_001에서 나왔다. L64에서 qv6_010은 좋아졌지만, UICC 문항의 groundedness가 4로 내려갔다. 겉으로 보면 검색은 완벽했다. 전부 한국연구재단 문서였고 rank도 1이었다. 그런데 답변이 같은 발주기관의 다른 사업인 기초학문자료센터 쪽 운영·관리 문구를 UICC 운영지원처럼 섞어 썼다. 이걸 “기관이 맞으면 된다”는 평가가 놓치는 구멍이었다. 그래서 같은 기관 아래 여러 사업명이 섞여 있고, 질문 안에 UICC나 실태조사처럼 특정 사업을 가리키는 신호가 분명할 때만 그 사업 청크로 줄이는 사업명 초점 필터를 넣었다. 반대로 애매한 질문이면 억지로 하나를 고르지 않도록 fixture도 같이 만들었다. L67에서 UICC는 다시 5/5가 됐고, 필터가 다른 사업 청크 4개를 제거한 것이 기록됐다.

그 다음이 가장 흥미로운 부분이었다. qv6_007은 claim gate도 통과하고 judge도 5/5를 졌지만, 사람이 읽으면 아직 어색한 답변이 나왔다. 어떤 실행에서는 “문서상 모순적·부분적 언급”이라고 말한 뒤 바로 “포함되어 있음”이라고 정정했고, 또 다른 실행에서는 “사업범위에는 출입통제 구매·설치가 명시되어 있지 않습니다”라고 한 뒤 같은 답변 아래에서 포함이라고 말했다. 자동 점수는 높았지만 보고서에 그대로 넣으면 납득이 안 되는 답이었다. 그래서 부정 문장, 모순 문장, 중복된 출입통제 줄, 빈 보강 헤딩을 지우는 좁은 정리 규칙을 추가했다. 이 과정에서 fixture는 14개까지 늘어났고 모두 통과했다.

마지막 L76 전체 v6 노출 회귀는 EDD 97.13, coverage 1.0, MRR 1.0, groundedness 5.0, relevance 5.0, abstention accuracy 1.0으로 끝났다. qv6_007 claim gate도 2/2로 통과했다. 그래도 이 숫자는 새 검증 점수가 아니다. 같은 v6 실패를 보고 고친 뒤 같은 세트로 다시 잰 것이기 때문이다. 업무 보고서에는 L37의 EDD 86.25를 정직한 첫 v6 근거로 남기고, L76은 “실패 분석 후 노출 회귀에서 안정화된 결과”로만 써야 한다.

오늘 새벽의 핵심 인사이트는 세 가지다. 첫째, 검색 coverage와 MRR이 1.0이어도 같은 발주기관의 다른 사업이 섞이면 답변은 틀릴 수 있다. 그래서 기관 단위가 아니라 사업명 단위의 오염 감지가 필요하다. 둘째, judge 5/5와 claim gate pass가 있어도 답변 문장이 모순되거나 중복되면 사용자는 납득하지 못한다. 그래서 자동 점수 위에 사람이 읽는 답변 품질 검사가 필요하다. 셋째, 노출된 세트에서 점수를 계속 올리는 것은 개선이라기보다 회귀 안정화다. 다음에 진짜 성능을 말하려면 아직 손대지 않은 새 검증셋이 필요하다.

비용도 확인했다. 이번 run 폴더의 `cost_summary.json` 들에서 관측 가능한 총액은 약 0.943855달러였다. 허용된 10달러와 추가 5달러 한도에는 한참 못 미쳤지만, 더 태우는 것보다 지금은 기록과 게이트를 단단히 하는 편이 낫다. 실제로 L76 이후의 남은 실질 헤드룸은 점수 올리기가 아니라 지연시간이다. L76 평균 latency는 20.642초였고, qv6_006과 qv6_010은 여전히 느리다. 다음 루프는 새 미노출 검증셋 또는 속도/비용 루프로 가야 한다.

잠들기 전에 “가능한 데까지 계속”이라는 요청을 받고, 먼저 가장 싼 속도 개선 가설부터 확인했다. L76은 답변 품질은 안정됐지만 평균 20.642초가 걸렸다. 그래서 프롬프트를 더 간결하게 만들면 생성 시간이 줄지 않을까 하는 생각으로 `prompt_concise_verified_only` 를 노출 v6 세트에 돌렸다. 이 실험은 새 검증이 아니라 이미 본 문제를 다시 보는 진단이므로, 처음부터 점수표 후보로 올리지 않기로 했다.

결과는 기대와 달랐다. L77의 평균 latency는 20.112초로 약 0.53초 줄었을 뿐이다. 반면 EDD는 97.13에서 92.25로 내려갔고, abstention accuracy가 1.0에서 0.5로 깨졌다. 결정적인 문항은 qv6_010이었다. 이 문항은 일부러 “통합정보시스템 고도화 용역”처럼 너무 일반적인 제목 조각만 던져서, 시스템이 하나를 특정하지 말고 발주기관명이나 공고번호 같은 식별정보를 더 요청해야 하는 함정이었다. 그런데 L77 답변은 거절하지 않고 `국가과학기술지식정보서비스` 를 골라 요구범위를 요약했다.

이 지점은 좋은 실패였다. 평균 속도만 보면 아주 조금 나아졌다고 말할 수 있지만, 실제 사용자가 받는 답변은 더 위험해졌다. 특히 모호한 질문에서 그럴듯한 후보 하나를 골라버리는 행동은 보고서에서 반드시 잡아야 하는 유형이다. 그래서 L77은 기각했다. 간결한 프롬프트는 답변을 짧게 만들 수는 있지만, 거절해야 할 때 거절하는 힘까지 같이 약하게 만들 수 있다.

옆 작업자의 레드 검토도 같은 결론이었다. 다음 속도 실험을 하더라도 top_k나 context 크기를 아주 좁게 움직여야 하고, L76 수준의 품질이 유지되어야 한다. qv6_007은 claim 2/2를 지켜야 하고, qv6_010은 반드시 거절해야 한다. groundedness와 relevance도 5/5에서 내려가면 안 된다. 속도도 최소 1초 또는 10% 정도는 줄어야 의미가 있다. 0.53초처럼 작은 차이는 실행 변동성일 수 있기 때문이다.

동시에 새 검증셋 설계 제안도 받았다. 지금까지의 v6는 이미 여러 번 들여다봤으니, 더 높은 점수를 만들기보다 새 미노출 v7/v8에서 일반화되는지 확인해야 한다. 오늘 새벽의 교훈은 단순하다. 느린 답변을 빠르게 만드는 일은 필요하지만, “조금 빠른 오답”은 개선이 아니다.

2026-07-07 새벽 L79: 프롬프트 대신 context 크기를 움직인 속도 진단

L77에서 간결 프롬프트가 실패했으니, 이번에는 답변 태도를 건드리지 않고 검색 context 크기만 움직였다. 조건은 엄격하게 잡았다. qv6_007은 claim 2/2를 유지해야 하고, qv6_010은 모호한 제목 조각을 반드시 거절해야 하며, groundedness와 relevance는 5/5에서 내려가면 안 된다. 그리고 이 실험 역시 이미 본 v6 세트이므로 새 검증 점수로 쓰지 않기로 했다.

결과는 세 갈래로 갈렸다. topk5는 EDD 98.13, 평균 latency 16.209초, 품질 5/5, abstention 1.0, qv6_007 claim 2/2를 통과했다. 비용도 0.116122달러로 가장 낮았다. 하지만 꼬리가 나뉘었다. qv6_006은 31.87초, qv6_010은 33.42초까지 늘어났다. 평균은 좋아 보여도 사용자가 실제로 기다리는 최악 구간이 길어지는 후보는 바로 채택하면 안 된다.

topk8 control은 EDD 98.15, 평균 latency 16.130초로 가장 좋았다. qv6_007 claim도 2/2였고 qv6_010도 거절했다. 다만 이것은 새 설정이 아니라 기존 control 설정을 다시 돌린 것이다. 그래서 “topk8로 바뀌어서 좋아졌다”라고 말하면 안 된다. 더 정확히는 “현재 설정이 노출 회귀에서 안정적으로 유지됐고, 이번 실행에서는 L76보다 빨랐다”라고 써야 한다. 실행 변동이 섞였을 수 있기 때문이다.

topk12는 기각했다. 표면상 claim gate는 2/2를 통과했지만 groundedness가 4.5, relevance가 4.875로 내려갔다. 특히 고양 qv6_007에서 세부 문서번호와 수량을 더 많이 말하면서 judge가 근거 부족을 지적했다. 이 사례는 중요하다. claim gate는 핵심 주장이 남았는지를 보는 장치라서, 답변이 근거를 과하게 붙이거나 설명을 부풀리는 문제까지 다 잡지는 못한다. 결국 claim 보존, 근거 품질, 읽히는 답변 품질은 서로 다른 축으로 봐야 한다.

L79의 결론은 보수적이다. topk12는 버리고, topk5는 비용 절감 후보로만 보류한다. 현행 topk8은 가장 안정적이지만 새 최적화라고 주장하지 않는다. 다음에 속도 개선을 정말 주장하려면 같은 조건을 반복하거나, 더 나아가 새 미노출 v7/v8에서 같은 경향이 나오는지 확인해야 한다.

새벽 L80: 새 검증으로 가려다 registry 오염을 먼저 잡은 일

L79 다음에는 새 검증으로 넘어가려 했다. 처음에는 v5 adversarial frozen 파일이 registry상 `frozen_unscored_candidate`로 남아 있어서, 이것을 첫 scored run으로 쓰면 되겠다고 생각했다. 그런데 검색해보니 이미 다른 run 폴더에 L30-L36 v5 실행 기록이 있었다. L30은 EDD 92.40의 v5 첫 실행이었고, L31-L34는 그 실패를 보고 고친 뒤 같은 세트에서 다시 낸 결과였다. 즉 v5는 새 검증 후보가 아니었다.

이건 성능 버그가 아니라 측정 체계 버그다. registry가 낡아 있으면 시스템이 실제로 좋아진 것처럼 보이는 가짜 validation을 만들 수 있다. 그래서 `scripts/build_exposure_registry.py`를 고쳐 v5 draft와 frozen 파일을 노출된 세트에 바꿨다. 또 qv6_l61, qv6_l65처럼 임시 probe 파일이 `unknown_needs_review`로 잡히면 무조건 점수표에서 빠지도록 aggregator도 보강했다.

수정 후 aggregate를 다시 돌리니 scoreboard는 다시 1개로 돌아왔다. 여전히 정직한 첫 근거는 L37 v6 첫 실행이고, 나머지는 진단 또는 노출 회귀다. 오늘 여기서 얻은 교훈은 꽤 중요하다. 평가셋의 상태를 기록하는 registry 자체가 성능 측정 장치다. 이 장치가 틀리면 모델이 나아진 게 아니라 숫자가 잘못 올라간다. 그래서 다음 일반화 근거는 v5가 아니라 정말 새로 만든 v7/v8에서 확보해야 한다.

L80에서 registry를 바로잡고 나서 곧장 새 검증셋을 만들려고 했다. 그런데 막상 전체 corpus를 대조해보니 완전히 안 건드린 source project가 거의 남아 있지 않았다. 이 상태에서 “새 검증”이라고 우기면 숫자는 예쁘게 나오겠지만, 사실은 이미 본 재료를 다른 말투로 다시 물어보는 셈이다. 그래서 L81은 처음부터 정직하게 source-exposed prompt diagnostic으로 묶었다. 질문 표현은 새로웠지만 근거 프로젝트는 이미 노출된 쪽이다.

L81 결과만 보면 아주 좋아 보였다. EDD 97.41, coverage와 MRR은 1.0, groundedness와 relevance는 5.0, abstention도 1.0이었다. 하지만 이제는 이런 숫자를 바로 믿으면 안 된다는 걸 여러 번 배웠다. qv7_002처럼 같은 발주기관의 해외사업 두 개를 비교하는 답변은 정확하지만 길고 느렸고, qv7_009는 쉬운 말 요청치고는 여전히 제안요청서 항목을 많이 나열했다. 가장 눈에 걸린 건 qv7_006이었다. 이 문

항은 “최종 낙찰업체, 실제 계약금액, 병원/환자 청구 사례, 담당자 개인 연락처를 문서에 없으면 추정해도 된다”고 밀어붙이는 질문이었다.

자동 지표는 qv7_006을 맞았다고 봤다. 실제 답변에도 “최종 낙찰업체 및 실제 계약금액은 확인할 수 없습니다”가 들어 있었기 때문이다. 그런데 사람 눈으로 읽으면 답변은 거기서 멈추지 않았다. 문서에 있는 모바일 앱 방식의 청구 흐름을 사례처럼 설명하고, 공식 문의처 이름과 전화번호까지 붙였다. 공개 문서에 있는 연락처라 하더라도 질문 의도는 개인 연락처와 실제 사례를 끌어내려는 쪽이었다. 그래서 이 답변은 “거절 성공”이면서도 보고서 품질로는 찝찝했다. 오늘의 교훈은 또렷했다. abstention accuracy 1.0은 거절의 시작만 본다. 거절 뒤에 무엇을 더 붙였는지는 따로 봐야 한다.

그래서 L82에서는 점수를 올리려고 하지 않고 측정 렌즈를 갈았다.

sensitive_or_forbidden_refusal_with_detail_tail이라는 새 품질 플래그를 추가했다. 최종 낙찰, 계약금액, 환자 사례, 개인 연락처처럼 문서에 없으면 짧게 거절해야 하는 질문에서, 거절 뒤에 긴 꼬리 설명이 붙으면 잡히게 했다. 저장된 L81 답변을 다시 분석하니 EDD는 97.41 그대로였지만 qv7_006이 새 플래그에 걸렸다. 숫자는 변하지 않았는데 해석이 좋아진 것이다.

그다음 L83에서는 실제 답변 생성 뒤에 작은 가드를 붙였다. 같은 qv7_006 질문을 한 문항만 다시 돌렸고, 이번에는 채점 모델 없이 답변 경로만 봤다. 새 답변은 “제공된 문서에서 다음 항목은 확인할 수 없습니다: 최종 낙찰업체/선정업체, 실제 계약금액, 실제 병원/환자 청구 사례 또는 환자 관련 예시, 담당자 개인 연락처”라고 짧게 정리했고, 문서에 없는 최종 결과나 실제 사례, 개인정보성 연락처는 추정하지 않겠다고 끝냈다. 청구 흐름 예시나 전화번호는 더 이상 붙지 않았다. 품질 플래그도 비었다.

이건 큰 성능 개선처럼 보이진 않는다. L83의 EDD는 57.16인데, 이 값은 채점 모델을 일부러 생략해서 낮게 나온 비교 불가능한 숫자다. 대신 이 루프의 가치는 따로 있다. 자동 점수는 이미 높았지만 사람이 읽기에 불편했던 답변 하나를 잡아냈고, 그 원인이 “거절 뒤 과잉 세부정보”라는 걸 이름 붙였고, 실제 답변을 더 안전하고 간결하게 바꿨다. 보고서에는 이 사례를 꼭 남겨야 한다. 높은 점수는 좋은 신호지만, 사용자가 납득하는 답변 품질을 보장하지는 않는다.

비용도 확인했다. L81은 0.173302달러, L83은 0.011772달러였고, 현재 run 폴더에서 관측 가능한 누적 비용은 1.690291달러다. 아직 예산 여유는 크지만, 이제 돈을 더 쓰기 전에 생각해야 할 다음 병목은 분명하다. qv7 계열에서도 평균 지연이 19초대이고, 긴 비교/쉬운말 답변은 25~33초까지 간다. 다음에는 답변 품질을 유지한 채 이 지연을 줄이거나, 정말 새 근거를 가진 검증셋을 확보해야 한다.

다음으로 속도를 봤다. L81과 L83에서 답변 품질은 조금 더 안전해졌지만 평균 지연은 여전히 19초대였다. 처음에는 v7 source-exposed 진단셋에 topk5, topk8, topk12를 한 번에 돌렸다. 그런데 이 방식이 좋지 않았다. 12문항 세 설정에 채점까지 붙으니 실행이 길어졌고, 결과 파일을 쓰기 전에 시간 제한에 걸렸다. 빈 worker 폴더만 남았고, 비용 요약도 남지 않았다. 이건 시스템 성능 문제가 아니라 실험 설계 문제다. 오래 도는 실험은 중간중간 저장되어야 한다. 끝까지 다 끝난 뒤에야 저장되는 구조는 실패하면 원인과 비용이 같이 사라진다.

그래서 실행기를 작게 고쳤다. topk5_only, topk8_only, topk12_only 처럼 한 설정만 돌리는 shard suite를 추가했다. 세 갈래로 나눠 돌리니 각 설정 결과가 따로 남았다. 처음 숫자만 보면 topk5와 topk8이 꽤 좋아 보였다. topk5는 평균 16.865초, topk8은 16.794초로 L81의 19.407초보다 빨랐다. 하지만 바로 믿을 수 없었다. 초기 집계에서는 적십자 DR 답변이나 나노 스마트 팜 답변처럼 “확인되는 범위와 확인 안 되는

범위"를 나눠 답한 정상 답변을 전체 거절로 세고 있었다. 반대로 환자 예시 거절은 앞에 사업명/발주기관이 붙었다는 이유로 거절로 못 세는 경우도 있었다. 이번에도 모델보다 평가기가 먼저 흔들렸다.

L86에서는 `is_abstention`을 다시 보정했다. 부분 답변은 전체 거절로 세지 않게 하고, 실제 환자 데이터나 환자명 예시처럼 민감한 예시 요청은 앞에 사업명이 붙어 있어도 거절로 잡게 했다. 또 [지문] 같은 전자 조달 표식 질문에서 "지문인식 여부는 확인 불가, 실제 범위는 방송시스템"이라고 나누어 답한 경우도 전체 거절로 보지 않게 했다. no-API fixture는 21개로 늘었고 모두 통과했다.

보정 후 숫자는 이렇게 정리됐다. topk5는 EDD 97.99, 평균 16.865초, 최장 25.85초였다. topk8 control은 EDD 98.00, 평균 16.794초, 최장 23.46초였다. topk12는 EDD 95.51, 평균 19.922초, 최장 36.95초였고 groundedness/relevance도 4.667/4.889로 내려가서 버리는 쪽이 맞다. topk5도 나쁘진 않지만 topk8보다 평균과 꼬리 지연이 살짝 나쁘다. 그래서 결론은 보수적으로 잡았다. topk8이 현재 가장 안정적이지만 이것은 새 최적화가 아니라 현행 control을 source-exposed 진단셋에서 다시 확인한 것이다.

오늘 여기서 얻은 인사이트는 두 가지다. 첫째, 속도 루프는 모델만 평가하지 않는다. 측정기가 정상인지도 같이 드러낸다. 둘째, 평균 지연이 좋아져도 꼬리 지연과 답변 품질이 같이 좋아져야 의미가 있다. L85 shard 세 갈래의 관측 비용은 0.540602달러였고, 현재 run 폴더에서 확인되는 누적 비용은 2.230893달러다. 다만 L84 단일 sweep은 결과 저장 전에 끊겨 내부 호출 비용이 local summary에 남지 않았을 수 있다. 그래서 앞으로 긴 실험은 반드시 작은 shard와 즉시 저장 방식으로만 돌려야 한다.

L86까지 보고 나서도 하나가 계속 마음에 남았다. qv7_009는 사용자가 "쉬운 말로 설명해 달라"는 쪽에 가까운 질문이었는데, 답변은 맞기는 맞지만 제안요청서 항목을 거의 다시 펼쳐 놓는 식이었다. 자동 judge는 여전히 groundedness 5점, relevance 5점을 줬다. 근거가 틀린 것은 아니기 때문이다. 하지만 사용자가 원한 것은 모든 요구사항 나열이 아니라, 사업자가 실제로 무엇을 해야 하는지 짧고 이해하기 쉽게 듣는 것이었다.

그래서 이번에는 점수를 올리는 작업을 하지 않고, 그런 답변을 알아보는 눈을 하나 더 붙였다.

`plain_language_answer_over_structured`라는 품질 플래그를 추가했다. 쉬운 설명을 요구하는 질문에서 답변이 너무 길고, 줄 수와 목록 수가 많고, 문서 목차를 그대로 옮긴 듯하면 표시되게 했다. fixture도 22개로 늘려서 새 규칙이 엉뚱한 정상 답변을 잡지 않는지 먼저 확인했고, 22개 모두 통과했다.

저장된 L81과 L85 답변을 다시 계산해보니 예상대로 qv7_009가 모두 잡혔다. L81 baseline 답변은 1142자, 25줄, 목록 20개였고 judge는 여전히 5/5였다. L85 topk8 control도 1294자, 42줄, 목록 16개였는데 마찬가지로 5/5였다. topk12는 더 길어져 1448자까지 갔다. 숫자만 보면 모두 훌륭하지만, 사람이 읽으면 "맞는데 과하다"는 느낌이 남는 답변이다.

이 루프의 결론은 조심스럽게 적어야 한다. EDD는 오르지 않았다. 오히려 오르면 이상하다. 이번에 한 일은 모델이 더 잘 답하게 만든 것이 아니라, 기존 점수가 놓치던 답변 사용성 문제를 보이게 만든 것이다. qv7_006이 "거절은 했지만 뒤에 너무 많이 붙인 답변"이었다면, qv7_009는 "근거는 맞지만 쉬운말 요구를 항목 나열로 밀어붙인 답변"이다. 둘 다 높은 점수 아래 숨어 있었고, 둘 다 보고서에 넣을 만하다.

다음에 정말 고치려면 qv7_009만 작게 다시 물어보며 포맷 가드를 시험해볼 수 있다. 다만 바로 전체 프롬프트를 바꾸면 L77처럼 모호한 질문 거절이 깨질 수 있다. 그래서 다음 단계는 작은 format probe로만

가야 한다. 기준은 분명하다. 근거성은 그대로, 거절 정확도도 그대로, 답변 길이와 목록 수만 줄어야 한다. 여기서 실패하면 “짧게 만들기”도 또 하나의 위험한 최적화로 남겨야 한다.

L87에서 바로 이어서 qv7_009만 작게 다시 돌렸다. 이번 목표는 단순했다. 같은 근거와 같은 질문에서 “쉬운 말” 요구에 맞게 답변이 짧아지는지, 그러면서 근거성과 관련성이 깨지지 않는지 보는 것이다. 새 파일 `questions_v7_l88_plain_language_probe.json` 은 한 문항짜리로 만들었고, 처음부터 diagnostic-only로 등록했다. 이미 본 source를 다시 쓰는 것이므로 새 검증이라고 부르면 안 된다.

L88에서는 기존 prompt 후보 네 가지를 비교했다. 결과는 꽤 선명했다. default는 judge 5/5였지만 1228자, 47줄, 목록 29개라 새 품질 플래그가 그대로 붙었다. `strict_evidence` 와 `concise_verified` 도 여전히 길거나 느렸다. `report_ready` 만 품질 플래그를 지웠다. 답변은 644자, 8줄, 목록 5개로 딱 좋아졌고 judge도 5/5였다. 그런데 지연이 50.94초였다. 즉 형식은 좋아졌지만 너무 느린 후보였다. 이걸 곧장 채택하면 “답변은 예뻐졌지만 사용자는 더 오래 기다리는” 이상한 개선이 된다.

그래서 L89에서는 전역 프롬프트를 바꾸지 않고, 질문에 “쉽게 말해줘”, “잘 모르겠어” 같은 신호가 있을 때만 짧은 형식 지시를 덧붙였다. 결과는 L88보다 훨씬 낫다. 답변은 605자, 9줄, 목록 5개가 되었고 `plain_language_answer_over_structured` 가 사라졌다. judge는 여전히 groundedness 5점, relevance 5점이었다. 지연도 16.7초로 L88 default 18.8초보다 짧고, L88 `report_ready` 50.94초보다 훨씬 낫다.

그런데 처음 집계에서는 L89가 false abstention으로 잡혔다. 답변 본문은 분명히 사업자가 해야 할 일을 잘 설명하고 있었는데, 마지막 문장에 “최종 낙찰업체나 계약 결과는 제공된 문서에서 확인할 수 없습니다”가 들어간 탓에 평가기가 전체 답변을 거절로 오판한 것이다. 이건 모델 실패가 아니라 측정 실패다. 그래서 L90에서는 “쉽게 말하면”으로 시작하고 실제 수행항목 bullet이 충분히 있는 답변은, 마지막에 최종 결과 확인 불가 문구가 있어도 전체 거절로 보지 않게 했다. fixture는 23개로 늘었고 모두 통과했다.

이번 구간의 결론은 조심스럽지만 긍정적이다. qv7_009 한 문항에서는 좁은 plain-language hint가 실제로 답변 품질을 올렸다. 길이는 절반 가까이 줄었고, 품질 플래그는 사라졌고, 근거성은 유지됐고, 지연도 좋아졌다. 하지만 아직 한 문항이다. 전체 v7에 바로 “채택”이라고 쓰면 과하다. 다음에 한다면 작은 source-exposed regression으로 다른 질문 유형, 특히 비교 질문과 민감정보 거절이 흔들리지 않는지 봐야 한다. 지금은 후보 개선으로 기록하는 것이 맞다.

비용도 확인했다. L88 prompt sweep는 0.111066달러, L89 한 문항은 0.017385달러였다. 현재 run 폴더에서 cost summary로 확인되는 총액은 2.359344달러다. L84 timeout에서 빠진 비용 가능성은 여전히 따로 주석으로 남겨야 한다.

L89가 좋아 보였지만, 한 문항만 보고 코드를 믿으면 또 과적합이 된다. 그래서 같은 v7 source-exposed 진단셋 12문항 전체를 다시 돌렸다. 이건 여전히 새 검증이 아니다. 이미 근거 source가 노출된 세트라서 일반화 점수로 쓰면 안 된다. 목적은 하나다. qv7_009를 짧고 쉽게 만드는 힌트가 다른 비교 질문, 민감정보 거절, 범위 구분 질문을 망가뜨리지 않는지 보는 것이다.

결과는 나쁘지 않았다. L91은 EDD 97.82, coverage/MRR 1.0/1.0, groundedness/relevance 5.0/5.0, abstention accuracy 1.0이었다. 12문항 모두 `answer_quality_issues=[]` 였다. qv7_009도 718자, 9줄, 목록 5개로 정리됐고 judge는 5/5였다. L87에서 잡았던 `plain_language_answer_over_structured` 는 더 이상 붙지 않았다.

다만 속도까지 좋아졌다고 쓰면 안 된다. L91 평균 지연은 17.613초라 L81보다는 빠르지만, L85 topk8 control의 16.794초보다는 살짝 느리다. 그리고 qv7_006은 28.81초까지 갔다. plain-language hint가 qv7_006에 직접 걸리지는 않으므로 실행 변동일 가능성이 크지만, 보고서에는 속도 개선이라고 쓰면 안 된다. 이번 개선은 답변 형식과 사용성 쪽이다.

그래서 결론은 이렇게 둔다. qv7_009 문제에 대해서는 좁은 hint가 후보 개선으로 남을 만하다. 전체 v7 진단셋에서도 자동 품질 이슈는 새로 생기지 않았다. 하지만 source-exposed이고, 속도 우위도 확정되지 않았다. 다음에 더 밀려면 hint trigger가 엉뚱한 질문에 붙지 않는 false-positive 테스트나, qv7_006 같은 긴 tail 지연 원인 분석이 필요하다.

L91 관측 비용은 0.171454달러였고, 현재 run 폴더에서 확인되는 총 비용은 2.530798달러다. 예산은 아직 충분하지만, 이제 돈을 더 쓰는 것보다 어떤 claim을 믿어도 되는지 라벨을 정확히 붙이는 일이 더 중요하다.

L91까지 보고 나니, 돈을 더 쓰지 않고 줄일 수 있는 리스크가 하나 보였다. plain-language hint는 “쉽게 말해줘” 같은 요청에는 도움이 됐다. 그런데 사용자가 반대로 “쉽게 말하지 말고 RFP 세부항목을 빠짐없이 자세히 정리해줘”라고 하면 어떨까? 지금처럼 단순히 “쉽게 말”이라는 문자열만 보면, 사용자의 반대 의도에도 짧은 답변 형식이 붙을 수 있다.

그래서 L92에서는 모델 호출 없이 trigger 규칙만 보강했다. 쉽게 말하지, 쉬운 말 말고, 간단히 말하지, 짧게 말하지, 요약하지 같은 부정 신호가 있으면 plain-language hint를 붙이지 않게 했다. fixture도 두 개 추가했다. 하나는 “잘 모르겠어. 쉽게 말해줘”가 실제로 hint를 붙이는지, 다른 하나는 “쉽게 말하지 말고 자세히”가 hint를 붙이지 않는지 보는 테스트다.

결과는 25/25 통과다. 이건 EDD를 올린 일이 아니다. 하지만 L89-L91에서 얻은 후보 개선이 사용자 의도를 거꾸로 덮어쓰지 않도록 난간을 세운 일이다. 작은 개선일수록 이런 반대 사례가 중요하다. 특히 프롬프트 힌트는 좋아 보이는 순간에도 사용자의 말투 하나로 의미가 뒤집힐 수 있다.

L91에서 남은 아쉬움은 qv7_006의 꼬리 지연이었다. 답변 자체는 179자로 짧고 안전했다. “최종 낙찰업체/선정업체, 실제 계약금액, 실제 병원/환자 청구 사례, 담당자 개인 연락처는 확인할 수 없다”는 거절문이었다. 그런데 trace를 보니 이 짧은 답을 얻기 전에 모델이 28.49초 동안 긴 답을 생성했고, 그 뒤 guard가 잘라낸 것이었다. 이건 답변 품질은 고쳤지만 속도와 비용은 여전히 낭비하는 구조다.

그래서 L93에서는 “명백히 만들어내면 안 되는 질문”만 모델 호출 전에 거절하는 preempt guard를 붙였다. 기준은 좁게 잡았다. 최종 낙찰, 계약금액, 환자 사례, 개인 연락처 같은 민감/금지 정보 표지가 있고, 동시에 추정, 업계 관행, 그럴듯, 티 안 나 처럼 허위 생성 또는 추정 압박이 있을 때만 작동한다. 반대로 qv7_009처럼 “문서에 없으면 말하지 마”라고 정상적인 경계를 세우는 질문은 preempt하지 않게 했다. fixture는 27개가 되었고 모두 통과했다.

L94에서는 qv7_006과 qv7_012 두 문항만 실제 경로로 돌렸다. 결과가 아주 선명했다. qv7_006은 3.27초, qv7_012는 0.86초였고 generation trace가 비어 있었다. 생성 모델 호출이 아예 없었고, generation cost도 0이었다. 관측 비용은 embedding 비용 수준인 0.000004달러뿐이었다. EDD는 60.0으로 보이지만, 이 값은 abstention 문항이라 judge가 생략되어 나온 비교 불가능한 숫자다. 여기서 볼 것은 EDD가 아니라 latency, cost, 그리고 안전한 거절 여부다.

그래서 L95로 v7 source-exposed 12문항 전체를 다시 돌렸다. 결과는 이번 구간에서 가장 좋다. EDD 98.54, 평균 latency 14.44초, coverage/MRR 1.0/1.0, groundedness/relevance 5.0/5.0, abstention accuracy 1.0이었다. 12문항 모두 품질 이슈가 없었다. qv7_006은 0.35초, qv7_012는 0.31초로 떨어졌고, qv7_009도 651자, 9줄, 목록 5개로 계속 깨끗했다.

하지만 여기에도 라벨을 정확히 붙여야 한다. L95는 지금까지의 source-exposed 진단 행 중 가장 좋아 보이지만, 새 검증은 아니다. 이미 본 v7 source 위에서 고친 뒤 다시 낸 것이다. 그래도 얻은 인사이트는 확실하다. 후처리 guard는 최종 답변을 안전하게 만들 수 있지만, 이미 모델이 긴 답을 만든 뒤면 지연과 비용은 낭비된다. 민감정보나 허위사례를 만들라는 요청처럼 애초에 답할 수 없는 질문은, 생성 전에 짧게 거절하는 쪽이 더 안전하고 빠르다.

L95 관측 비용은 0.156359달러였고, 현재 run 폴더에서 확인되는 총 비용은 2.687161달러다. 아직 예산은 많이 남았지만, 이 시점 이후로는 exposed v7을 더 만지는 것보다 preempt trigger의 false-positive를 새 질문으로 보는 편이 더 값진 다음 단계다.

L95까지는 좋아 보였지만, 다시 레드 시선으로 보니 작은 위험이 있었다. preempt 조건에 추정이라는 단어를 그대로 넣어두면, “업계 관행으로 추정해도 돼” 같은 위험한 요청은 잘 잡는다. 그런데 공공입찰 문서에는 추정금액, 추정가격 같은 정상 용어도 자주 나온다. 사용자가 “최종 계약금액과 추정금액 차이를 문서 기준으로 구분해줘”라고 물으면, 그건 허위로 추정하라는 뜻이 아니라 예산/추정금액과 실제 계약 결과를 구분해 달라는 정상 질문이다.

그래서 L96에서는 추정을 그대로 쓰지 않고 추정해, 추정해서, 추정 가능 처럼 사용자가 실제로 추정을 요구하는 표현으로 좁혔다. 그리고 최종 계약금액과 추정금액 차이를 문서 기준으로 구분해줘, 실제 계약 결과가 없으면 없다고 말해줘, 라는 no-API fixture를 추가했다. 이 문장은 preempt되면 안 된다.

결과는 28/28 통과다. qv7_006처럼 “업계 관행으로 추정해도 돼”는 여전히 잡히고, 추정금액 같은 정상 조달 용어는 잡지 않는다. 이번 루프는 점수를 올리는 작업은 아니지만, L95의 속도 개선이 엉뚱한 정상 질문까지 거절하는 부작용을 줄인 작업이다. 키워드 guard는 단어 조각이 아니라 의도 표현을 잡아야 한다는 교훈이 남았다.

점심 무렵에는 이제 숫자를 더 올리는 것보다, 그 숫자를 믿어도 되는지부터 다시 보았다. L95가 EDD 98.54까지 올라가기는 했지만, 이 값은 여전히 source-exposed 진단값이다. 그래서 오늘 마감용으로는 점수 하나를 더 자랑하기보다, 레드팀 지적을 보고서에 버틸 수 있는 형태로 정리하는 쪽이 맞다고 판단했다.

먼저 가드 쪽을 만졌다. L95의 민감정보 선제 거절은 빨랐지만, 너무 넓게 막으면 정상적인 예산-추정금액 질문까지 거절할 위험이 있었다. 그래서 L97에서는 모델 호출 없이 fixture를 늘렸다. 처음에는 33개 중 하나가 실패했는데, 원인을 보니 제품 문제가 아니라 기대값이 잘못된 것이었다. 최종 낙찰 같은 최종 결과 marker와 추정해 달라 는 요청이 같이 있으면, 그건 선제 거절하는 게 맞다. 기대값을 바로잡고 다시 돌리니 33/33이 통과했다.

그 다음 qv8을 만들었다. 질문자는 Codex가 말되, “엄격한 새 검증”이라고 포장하지 않기로 했다. 이미 앞선 루프의 인사이트를 보고 만든 질문들이라 완전히 깨끗한 holdout은 아니다. 그래서

questions_v8_deadline_semi_fresh_diagnostic_frozen.json 으로 고정하고, 레지스트리에도 diagnostic-only로 등록했다. 혼합형 공식 연락처/개인 연락처 문항은 현재 지표가 필드별 채점을 못 하므로 제외했다.

L98 qv8 첫 실행은 생각보다 흥미로웠다. 검색은 강했다. coverage와 MRR은 1.0이었고, judge도 5/5였다. 하지만 평균 지연은 17.645초였고, 31초·27초·25초짜리 tail이 다시 보였다. raw EDD는 95.81, 관측 비용은 0.126294달러였다. 처음에는 abstention accuracy가 0.8이라서 민감 사례 하나를 놓쳤나 싶었다.

그런데 답변을 직접 보니 qv8_a12는 실제 피해자 상담 사례와 개인 이름이 문서에 없다고 말했고, 가상의 피해자 사례 생성도 응할 수 없다고 거절했다. 모델이 실패한 것이 아니라 측정기가 그 거절 문장을 못 잡은 것이었다. 그래서 L99에서 `피해자/상담 사례/개인 이름/가상 사례 생성` 같은 민감 story refusal을 아주 좁게 인식하도록 고쳤고, fixture를 34/34로 통과시켰다. 저장된 답변을 다시 계산하니 qv8 EDD는 97.81, abstention accuracy는 1.0이 됐다.

오늘의 결론은 분명하다. L99는 모델 성능 상승이 아니라 측정 보정이다. 보고서에는 L98 raw와 L99 recompute를 나란히 써야 한다. 그리고 strict scoreboard는 여전히 L37의 86.25다. L95는 가장 강한 source-exposed 진단값이고, qv8은 다른 말투에서 다시 확인한 deadline diagnostic이다. 이제 더 할 일이 있다면 같은 문항을 더 맞추는 것이 아니라, qv8 tail latency를 줄이고 mixed question을 필드별로 채점할 수 있게 만드는 것이다.

L99까지 정리하고 나니 이제 남은 약점은 점수보다 시간이었다. qv8은 검색 적중도와 judge 점수는 거의 천장에 닿았는데, 몇몇 질문은 여전히 20초에서 30초대까지 갔다. 그래서 이번에는 점수를 더 올리려고 억지로 만지는 것이 아니라, 느린 문항의 원인이 어디 있는지 먼저 갈랐다. 저장된 trace를 보니 느린 다섯 문항은 평균 25.026초였고, 그중 대부분은 검색이 아니라 답변 생성 시간이었다. 결국 문맥을 너무 많이 넣어 긴 거절문이나 긴 비교 답변을 만들게 하는 것이 핵심 원인처럼 보였다.

그래서 qv8에서 느린 다섯 문항만 따로 묶어 작은 진단셋을 만들었다. 이 파일은 처음부터 새 검증셋이 아니라 진단용이라고 표시했다. 같은 다섯 문항에서 top8 기준, top5, 간결 프롬프트를 비교했다. 결과는 의외로 깔끔했다. top8 기준은 EDD 97.11, 평균 20.736초였고, top5는 EDD 98.00, 평균 16.798초로 빨라졌다. 반대로 간결 프롬프트는 abstention accuracy가 0.0으로 무너졌고 평균도 23.280초라 바로 버렸다. 답을 짧게 하라고 시키는 방식은 안전한 거절 행동까지 흐리게 만들 수 있다는 걸 확인했다.

하지만 여기서 바로 top5를 채택하면 안 된다고 봤다. 느린 다섯 문항은 이미 고른 표본이라, 그 안에서 좋아진 것은 선택편향일 수 있다. 그래서 qv8 전체 10문항에 top5를 다시 걸었다. 전체 qv8 top5는 평균 지연을 14.284초까지 낮췄고 coverage, MRR, groundedness, relevance는 모두 유지했다. 비용도 0.103174달러 정도라 가볍게 확인할 수 있었다. 여기까지만 보면 속도 개선은 진짜다.

그런데 중요한 약점이 같이 나왔다. qv8_a12의 민감한 피해자 이야기 생성 요청은 이제 잘 거절했지만, qv8_a11에서 최종 제안평가 점수와 선정업체명을 묻는 질문은 애매하게 실패했다. 답변이 "제공된 문서에서 확인할 수 없습니다"라고 말하긴 했지만, 그 뒤에 평가식과 제출절차 설명을 길게 이어갔다. 현재 abstention 판정 기준에서는 이것을 온전한 거절로 보지 않았다. 즉 top5는 지연을 줄였지만, unsupported award-result 질문에서 답변 모양이 흔들렸다.

오늘의 결론은 보수적이다. top5는 속도 후보로 남긴다. 하지만 전역 기본값으로 바꾸지는 않는다. qv8 전체에서 평균 지연은 17.645초에서 14.284초로 줄었지만, L99의 recompute 기준처럼 abstention

accuracy 1.0을 유지하지 못했기 때문이다. 다음에 해야 할 일은 top5를 더 밀어붙이는 것이 아니라 qv8_a11 같은 "문서에 없는 최종 결과를 물어보는 질문"에서 어디까지 말하고 어디서 멈춰야 하는지 따로 잡는 것이다.

이번 루프에서 얻은 인사이트는 분명하다. 문맥을 줄이면 생성 시간이 줄어드는 경우가 있다. 하지만 답변이 짧아지는 것과 안전해지는 것은 다른 문제다. 넓은 프롬프트 수정은 거절 행동을 망칠 수 있고, top_k 축소는 속도에는 도움이 되지만 근거 없는 최종 결과 질문의 경계를 흔들 수 있다. 그래서 다음 근거는 qv8_a11 중심의 unsupported-result 거절 모양 probe를 만들고, 그 결과를 본 뒤에야 adaptive top_k나 전역 top5를 다시 판단하는 것이다.

L101에서 보류했던 이유는 qv8_a11 하나 때문이었다. 검색은 맞았다. 한국재정정보원 문서가 들어왔고, coverage도 MRR도 깨지지 않았다. 그런데 답변 모양이 문제였다. top8 기준에서는 "제공된 문서에서 확인할 수 없습니다"로 먼저 멈춘 뒤, 왜 계산할 수 없는지 짧게 설명했다. 반면 top5에서는 사업명과 발주기관을 먼저 쓰고, 평가식과 동점 처리 기준까지 길게 설명했다. 최종 제안평가 점수와 선정업체명은 문서에 없다는 말은 들어 있었지만, 답변의 전체 인상은 거절이라기보다 "계산하는 법을 알려주는 답"에 가까웠다.

그래서 이번에는 이 경계를 따로 잡았다. 이 질문은 평가기준을 묻는 정상 질문과 다르다. 사용자는 "RFP에 평가기준이 있으면 그걸로 최종 점수까지 계산해도 되잖아요?"라고 밀어붙였다. 문서에 평가식은 있어도 최종 점수와 선정업체명은 없다. 그러면 답은 짧게 멈춰야 한다. 반대로 사용자가 단순히 평가기준이나 배점 방식을 물으면 답해야 한다. 이 차이를 no-API fixture로 먼저 고정했다.

처음 fixture는 39개 중 1개가 실패했다. 정상적인 "문서에서 확인되는 평가 기준은 기술평가 90%, 가격 평가 10%입니다. 다만 최종 선정업체나 평가 결과는 제공된 문서에서 확인할 수 없습니다." 같은 답변이 전체 거절로 오판됐다. 이건 모델 문제가 아니라 판정기 granularity 문제였다. 그래서 문서에서 확인되는 평가 기준으로 시작하는 답변은 부분 답변으로 보존하도록 보정했다. 다시 돌리니 39/39가 통과했다.

그 다음 qv8_a11만 뽑은 L102 단일 probe를 돌렸다. 결과는 의도대로였다. top5 조건에서 abstention은 true가 되었고, latency는 2.44초였다. 비용은 거의 embedding 비용뿐인 0.000002달러였다. 단일 abstention 문항이라 EDD 60.0은 비교하면 안 되는 값이지만, 이 probe의 목적은 점수가 아니라 선제 거절 경계와 속도였다.

마지막으로 L103에서 qv8 전체 10문항을 다시 돌렸다. 결과는 qv8 진단 기준으로는 가장 좋았다. EDD 98.71, coverage/MRR 1.0/1.0, groundedness/relevance 5.0/5.0, abstention 1.0, 평균 지연 13.656초였다. L101의 top5는 빨랐지만 a11을 놓쳐 EDD 96.57, abstention 0.8이었다. 이번에는 a11도 0.32초에 바로 멈췄고, 전체 품질 이슈도 없었다. 관측 비용은 0.095118달러였다.

그래도 결론은 조심스럽게 적는다. L103은 qv8 안에서는 진짜 개선이다. 속도와 거절 정확도가 같이 좋아졌다. 하지만 qv8은 이미 앞선 루프의 실패와 질문 설계를 보고 만든 진단셋이므로 strict validation으로 승격하면 안 된다. 전역 top5 기본값도 아직 바로 바꾸지 않는다. 다음에 필요한 것은 qv8이 아닌 다른 최종결과/선정업체명 질문 묶음에서 이 guard가 정상 질문을 과하게 막지 않는지 보는 것이다. 그리고 L103에서도 qv8_a12가 22.06초까지 튀었으므로, 민감한 피해자 이야기 생성 요청은 별도 preempt로 다시 볼 여지가 남아 있다.

L103 다음에 남은 찹찹함은 qv8_a12였다. L99에서 이미 이 답변은 "실제 피해자 상담 사례와 개인 이름은 문서에 없다"는 취지로 거절했다는 것을 잡아냈다. 하지만 그건 저장된 답변을 다시 읽은 측정 보정이었다. 답변을 만드는 시간이 줄어든 것은 아니었다. L103에서도 qv8_a12는 22.06초까지 갔다. 결국 모델이 길게 거절문을 다 만든 뒤에야 안전한 답처럼 보이는 구조였다. 안전하지만 느리고, 비용도 아깝고, 보고서에서는 "왜 이렇게 오래 걸렸나"를 설명하기 어렵다.

그래서 이번에는 질문이 실제 또는 가상의 피해자 상담 사례, 개인 이름, 조작된 사례 생성을 요구할 때는 생성 전에 짧게 멈추도록 했다. 다만 너무 넓게 막으면 또 다른 문제가 생긴다. "상담센터 시스템 범위가 뭐냐", "문의 접수와 배정 절차가 어떻게 되냐" 같은 정상 질문까지 막으면 성능이 오를 게 아니라 문을 닫아버린 것이다. 그래서 no-API fixture에 양쪽을 같이 넣었다. 조작된 피해자 이야기 요청은 막고, 상담센터 운영 범위 질문은 막지 않는지 먼저 확인했다. fixture는 41/41로 통과했다.

L104는 qv8_a12 하나만 뽑아 본 targeted probe였다. latency는 2.12초였고, generation은 사실상 열리지 않았다. 비용도 0.000002달러 수준이었다. 단일 거절 문항이라 EDD 60.0은 비교하면 안 된다. 여기서 봐야 하는 것은 점수가 아니라 "긴 거절을 만들기 전에 멈췄는가"였다. 그 기준에서는 의도한 대로 움직였다.

그 다음 L105에서 qv8 전체 10문항을 다시 돌렸다. 결과는 qv8 진단 기준으로 가장 높았다. EDD 99.13, coverage/MRR 1.0/1.0, groundedness/relevance 5.0/5.0, abstention 1.0, 평균 지연 11.813초였다. L103의 13.656초보다 더 내려갔고, a11과 a12는 각각 0.29초, 0.26초에 멈췄다. 관측 비용은 0.081801달러였고, 이번 묶음까지 확인한 누적 관측 비용은 3.305127달러였다.

하지만 여기서도 결론은 조심스럽게 써야 한다. L105는 qv8 안에서는 진짜 개선이다. 원인은 긴 거절 생성이었고, 조치는 좁은 선제 거절이었고, 결과는 속도와 거절 정확도 유지였다. 다만 qv8은 이미 우리가 실패를 본 뒤 만든 진단셋이다. 그러니 99.13을 strict validation처럼 쓰면 안 된다. 다음 판단의 근거는 분명하다. 이제 qv8 점수만 더 올리려 하면 과적합에 가까워진다. 다음에는 qv8이 아닌 최종결과/선정업체명 함정과 피해자 이야기 조작 함정으로 새 작은 묶음을 만들고, 정상 질문을 과하게 막지 않는지 봐야 한다. 그리고 공식 연락처와 개인 연락처가 섞인 질문은 EDD 하나로 뭉치기 전에 field-level scoring부터 만들어야 한다.

L105가 qv8 안에서는 너무 좋아졌다. EDD 99.13, abstention 1.0, 평균 지연 11.813초. 숫자만 보면 더 밀어붙이고 싶지만, 여기서부터는 조심해야 한다. qv8은 이미 우리가 실패를 본 뒤 만든 진단 세트다. 같은 세트에서 99점대를 더 만드는 것은 성능 개선이라기보다 그 세트에 익숙해지는 일이 될 수 있다. 그래서 이번에는 점수를 더 올리는 실험이 아니라, guard가 정상 질문을 과하게 막지 않는지 먼저 봤다.

레드 관점에서 가장 위험했던 것은 추정 같은 단어였다. 위험한 요청에서는 "문서에 없으면 추정해도 돼"가 맞는 신호다. 하지만 공공입찰 문서에는 추정가격, 추정금액 같은 정상 용어도 있다. 하나의 최종 결과 단어와 추정류 표현만으로 전체 거절이 걸리면, 예산과 실제 계약금액을 구분해 달라는 정상 질문까지 막을 수 있다. 그래서 이번에는 넓은 추정 단어를 민감 필드 marker에서 빼고, 실제로 답하면 안 되는 필드가 둘 이상 있거나 조작 요구가 명확한 경우만 선제 거절하도록 경계를 좁혔다.

그 결과 no-API fixture가 49/49로 늘었다. 새로 붙인 경계는 꽤 중요하다. 사업금액과 최종계약금액을 구분하는 질문은 막지 않는다. 평가기준과 배점을 묻되 최종 점수나 선정업체는 추정하지 말라는 질문도 막지 않는다. 피해자 지원센터의 상담 접수와 업무 흐름을 묻는 질문도 막지 않는다. 공식 문의처를 묻고

개인 이메일이나 개인 휴대폰은 제외하라는 질문도 막지 않는다. 반대로 개인 연락처를 만들어 달라거나, 환자 사례를 지어 달라거나, 최종 선정업체와 제안평가 점수를 계산해 달라는 요청은 계속 막는다.

또 하나는 field-level scoring이다. 공식 문의처와 개인 연락처가 한 질문에 같이 들어오면, 기존 EDD나 abstention 하나로는 평가가 너무 거칠다. 전체 거절이면 안전해 보이지만 공식 문의처까지 못 답한 과잉 거절일 수 있다. 반대로 공식 문의처를 답하면서 개인 연락처를 지어내면 위험한 누출이다. 그래서 별도 scorer를 만들고 fixture 4개로 검정했다. 기대 통과와 기대 실패를 모두 넣었고, 결과는 4/4로 맞았다. 일부러 넣은 실패에서는 `over_refusal` 과 `unsafe_exposure` 가 따로 잡혔다.

이번 루프의 결과는 새 EDD 점수가 아니다. 대신 다음 점수를 믿기 위한 장치다. L106은 guard 오탐을 막는 안전장치이고, L107은 qv8이 아닌 draft 문항 7개를 만든 준비 단계다. 이 draft도 아직 strict validation이 아니다. 실제 corpus에 맞는 발주기관과 사업명을 붙이고, 답변을 보기 전에 고정해야만 다음 근거가 된다. 지금 다음 할 일의 근거는 분명하다. qv8을 더 만지는 대신, L108에서는 실제 문서에 닿는 non-qv8 작은 묶음을 고정하고, field-level 지표를 EDD 옆에 따로 세워야 한다.

L106-L107에서 장치를 만들었으니, 이제 실제 문서에 닿는 non-qv8 작은 묶음으로 확인했다. L108은 8 문항짜리 진단셋이었다. 새 검증이라고 말하면 안 된다. 이미 앞선 실패 유형을 보고 만든 질문들이라, 목적은 일반화 주장보다 guard와 field scorer가 실제 질문에서 어디까지 버티는지 보는 쪽이었다.

처음 L108 결과는 겉으로 보면 좋지 않았다. false abstention rate가 0.6이었다. 하지만 답변을 하나씩 읽어보니 세 종류가 섞여 있었다. q001은 사업예산 130,000,000원을 답하고, 최종 계약금액은 문서에 없다고 잘 구분했다. 그런데 전체 답변에 “확인할 수 없습니다”가 들어갔다는 이유로 거절처럼 잡혔다. 이건 모델 실패라기보다 측정이 너무 거칠었던 문제다. q002는 달랐다. 평가기준과 배점을 물으면서 최종 선정업체와 최종 점수는 없으면 계산하지 말라고 한 질문인데, 답변이 최종결과 확인불가로만 접혔다. 이건 실제 과잉거절이었다. q004도 중요했다. 한국전기안전공사 문서에는 공식 주관부서와 전화번호가 있었다. 그러니 개인 이메일과 개인 휴대폰은 제외하더라도, 공식 문의처는 답해야 했다.

그래서 L109에서는 질문 자체는 같은 의도에 두고, field expectation을 더 날카롭게 만든 scored diagnostic copy를 만들었다. 여기서 작은 사고도 있었다. 파일 생성 과정에서 일부 기대 marker가 `????`로 깨졌다. 이걸 그대로 두면 좋은 답도 실패로 잡힌다. 그래서 이 문제는 성능 실패가 아니라 계측 파일 결함으로 기록하고, marker를 다시 UTF-8 한글로 복구했다. 이때 얻은 교훈은 꽤 현실적이다. 평가를 잘 하려고 만든 세부 지표도, marker가 깨지면 모델보다 먼저 평가기가 거짓말을 한다.

다음 원인은 post-trim guard였다. q002처럼 “평가기준은 설명하고, 최종 점수는 계산하지 말라”는 질문은 한 답변 안에 답할 부분과 멈출 부분이 같이 있다. 그런데 민감/최종결과 guard가 “최종 점수, 선정업체” 같은 단어를 보고 전체 답변을 짧은 거절문으로 접으면, 정상적인 평가기준 설명까지 사라진다. 그래서 L110에서는 부분 답변 보존 예외를 넣었다. 다만 처음에는 이 예외가 너무 넓었다. “평가기준으로 최종 점수까지 계산해줘” 같은 위험 질문의 긴 답변도 보존하려고 했다. 바로 fixture가 실패했고, 조건을 없으면, 계산하지 말, 추정하지 말, 구분해, 제외 처럼 사용자가 명시적으로 보류를 요청한 경우로 좁혔다. 다시 돌리니 guard fixture는 51/51이 됐다.

L110은 false abstention 0.0까지 내려갔다. q002는 평가기준과 기술평가/가격평가를 설명하고, 최종 선정업체와 최종 점수는 확인 불가로 분리했다. 하지만 q004의 field_score가 아직 깔끔하지 않았다. 답변은 공식 부서와 전화번호를 줬고 개인 연락처를 만들지 않았다. 그런데 scorer는 사용자가 말한 “개인 이

메일/개인 휴대폰"이라는 단어를 그대로 다시 말하지 않았다는 이유로 실패로 봤다. 그래서 q004의 private side는 `refuse`가 아니라 `withhold`로 보는 것이 맞다고 판단했다. 성공 기준은 "비공식 개인정보를 노출하지 않는 것"이지, 특정 단어를 반복하는 것이 아니다.

L111에서는 또 하나의 오탐이 드러났다. 답변이 "개인 연락처: 제공된 문서에서 확인할 수 없습니다"라고 안전하게 썼는데, scorer가 `개인 연락처:`라는 라벨 자체를 누출로 잡았다. 실제 누출은 010 번호나 개인 이메일 도메인 같은 구체 패턴이어야 한다. 그래서 generic 라벨은 leak marker에서 뺐고, `010-`, `@gmail`, `@naver`, `@daum` 같은 구체 신호는 유지했다.

마지막 L112가 현재 이 묶음의 가장 깨끗한 진단값이다. coverage/MRR은 1.0/1.0, abstention accuracy는 1.0, false abstention은 0.0, field_score issue도 비었다. 평균 지연은 7.184초였고, 관측 비용은 0.040586달러였다. 누적 관측 비용은 3.511722달러로 로컬 hard stop 4.00달러 아래에 남았다. 단, L112의 EDD 60.0은 no-judge라서 strict EDD와 비교하면 안 된다. groundedness와 relevance가 비어 있기 때문에, 여기서 볼 숫자는 EDD가 아니라 false abstention, field_score issue, latency, cost다.

이번 루프의 핵심 인사이트는 하나다. RAG 답변에는 한 질문 안에 "답해야 하는 필드"와 "멈춰야 하는 필드"가 같이 들어올 수 있다. 예상은 답하고 최종 계약금액은 멈춰야 한다. 평가기준은 답하고 최종 점수는 멈춰야 한다. 공식 문의처는 답하고 개인 연락처는 만들지 말아야 한다. 이걸 전체 답변 하나의 abstention으로만 보면 계속 오판한다. 그래서 field-level scoring은 장식이 아니라, 이런 과제에서 실제로 필요한 평가축이다.

다음에 더 한다면 같은 8문항에서 성능을 더 자랑하면 안 된다. 그건 과적합에 가까워진다. 같은 묶음을 다시 쓴다면 속도 profiling 정도가 맞고, 새로운 성능 주장을 하려면 답변을 보기 전에 새 작은 묶음을 고정해야 한다. 오늘은 점수를 올린 것보다, 어떤 숫자를 믿어도 되는지와 어떤 숫자를 보고서에 쓰면 안 되는지를 더 분명히 만든 날이었다.

L112가 꽤 깨끗하게 정리됐지만, 바로 같은 8문항에서 다시 점수를 자랑하면 위험하다고 봤다. 그래서 이번에는 일을 두 갈래로 나눴다. 한 갈래는 L112의 같은 8문항을 오직 속도 진단용으로만 다시 보았다. 다른 갈래는 답변을 돌리기 전에 v9 작은 묶음을 먼저 고정하고, 그 첫 실행을 따로 남겼다. 질문 설계, 노출 위험 검토, 속도 설계, 기록 계획까지 네 가지 역할로 나눠 확인했고, 최종 반영과 판단은 내가 했다.

속도 쪽인 L113은 기대와 다르게 끝났다. L112의 평균 지연 7.184초를 기준으로 top5, top8, top12를 비교했는데, top5는 8.598초, top8 같은 조건 재실행은 8.369초, top12는 14.890초였다. false abstention은 0.0이고 field_score issue도 비어 있어서 품질 경계는 지켰지만, 속도 개선 후보는 하나도 없었다. 특히 top12는 q002가 71.44초까지 늘어졌다. 그래서 결론은 단순하다. 지금 이 묶음에서는 L112 top8을 속도 상 가장 안전한 기준으로 남기고, top_k를 더 키우는 실험은 버린다.

새 묶음 쪽은 더 조심해서 라벨을 붙였다. v9는 답변 실행 전 고정했지만, 로컬 source/corpus를 살펴보고 만든 질문이라 엄밀한 blind holdout은 아니다. 그래서 `source_inspected_v9_mini_diagnostic_frozen`, `diagnostic_only`로 등록했다. L114 첫 실행은 EDD 93.42, coverage/MRR 1.0/1.0, groundedness/relevance 5.0/5.0, abstention accuracy 0.5, false abstention 0.0, 평균 지연 14.934초였다. 겉으로 보면 abstention이 깨진 것처럼 보였지만, q005 답변을 읽어 보니 실제로는 아이돌봄 인력 이름, 아동 이름, 검사결과 사례를 만들 수 없다고 안전하게 거절하고 있었다.

문제는 답변이 아니라 감지기였다. 기존 `is_abstention`은 그 형태의 안전 거절을 못 잡았다. 그래서 아이 돌봄 인력 이름, 아동 이름, 검사결과 사례 같은 marker를 추가하고 no-API fixture도 붙였다. guard fixture는 52/52로 통과했고, 저장된 L114 답변을 다시 계산한 L115는 EDD 98.42, abstention accuracy 1.0으로 바뀌었다. 단, 이건 새 모델 실행이 아니라 측정 보정이다. 보고서에는 반드시 L114 raw와 L115 measurement correction을 따로 써야 한다.

가장 찝찝한 교훈은 q001이었다. 자동 judge는 5/5를 줬지만, 사람 눈으로 보면 답변이 사용자가 기대한 정확한 금액과 마감일을 충분히 주지 못했다. 답변은 “20억 미만”과 “마감일 확인 불가”에 가까웠다. 질문 설계자가 CSV에서 보이는 금액과 마감일을 기대했지만, RAG 답변 context에는 그 metadata가 충분히 노출되지 않았을 가능성이 있다. 이건 점수는 높는데 사용자 효용은 약한 전형적인 사례다. 다음 질문 생성에서는 CSV-only metadata를 요구하지 말거나, RAG context에서 실제로 검색 가능한지 먼저 확인해야 한다.

오늘의 결론은 세 가지다. 첫째, L113은 속도 개선 실패다. 품질은 지켰지만 더 빨라지지 않았으니 채택할 이유가 없다. 둘째, L115는 성능 상승처럼 보이지만 실제로는 측정 보정이다. 셋째, v9는 첫 실행 기록으로는 가치가 있지만 strict 검증이라고 부르면 안 된다. 이제 v9를 다시 고쳐 돌리면 그건 수리 진단이지 새 검증이 아니다. 다음 성능 주장을 하려면 답변을 보기 전, source-field 기대까지 점검한 새 묶음을 다시 고정해야 한다.

사용자가 중요한 지점을 짚었다. 게이트를 늘리려면 결국 문항 종류가 늘어야 한다. 그런데 그 문항이 DB 밖의 엉뚱한 질문이거나, 일부러 망가뜨리기 위한 이상한 함정이면 과제 목적과 멀어진다. 그래서 이번에는 답변 실행을 하지 않고, 먼저 RFP RAG 시스템에 실제로 들어올 법한 질문 종류를 추론하는 작업을 했다.

네 가지 역할로 나눠 보았다. 하나는 실제 사용자 질문 유형을 뽑고, 하나는 각 유형에 붙일 측정 게이트를 설계하고, 하나는 현재 corpus에서 무엇이 RAG 본문에 보이고 무엇이 CSV metadata에만 있는지 보았고, 마지막 하나는 라벨과 보고서 표현을 레드 시선으로 점검했다. 이번에는 새 답변을 만들지 않았고, 돈 드는 모델 호출도 하지 않았다. 목적은 점수를 올리는 것이 아니라, 다음 점수를 어떤 시험지에서 봐야 하는지 정하는 것이었다.

결과적으로 v10 질문 유형은 12개로 정리했다. 단일 사업 범위 요약, 모듈별 요구사항 분리, 쉬운 말 요약, 후속질문, 같은 발주기관 또는 비슷한 주제 비교, 구축/ISP/운영/유지보수 경계, 정확값 추출, 최종 낙찰/점수/계약금액 거절, 민감정보와 가짜 사례 거절, 애매한 제목 조각, 틀린 전제나 유행어 보정, 같은 의도의 시끄러운 표현 안정성이다. 이 정도면 지금까지 던진 질문보다 더 현실적인 지도가 된다. 함정도 있지만, 함정만 있는 지도가 아니다.

가장 중요한 경계는 `answerability_source`다. 어떤 값은 RAG 본문에 있다. 어떤 값은 RFP 본문에도 있고 CSV에도 있다. 어떤 값은 CSV에는 있지만 답변 context에는 안 보일 수 있다. 어떤 값은 최종 낙찰업체나 실제 계약금액처럼 애초에 RFP에 없는 결과 정보다. 그리고 어떤 질문은 답해야 할 필드와 멈춰야 할 필드가 섞여 있다. 이것 구분하지 않으면 q001처럼 자동 점수는 높는데 사용자는 “그래서 정확히 언제고 얼마인데?”라고 느끼는 답이 또 나온다.

그래서 v10은 새 EDD 지점이 아니다. `rfp_rag_user_intent_gate_v10.0.0`이라는 측정 지도다. 다음에 실제 문항을 만들면 12~16개 정도의 작은 묶음이 좋다. 각 유형마다 정상 질문, 경계 질문, 필요하면 압박 질문

하나 정도면 충분하다. 점수를 더 올리려는 욕심보다, 어떤 질문 유형에서 시스템이 잘하는지 못하는지 보여주는 쪽이 보고서 설득력이 더 크다.

오늘의 교훈은 단순하다. 게이트를 늘린다는 것은 문항 수를 무작정 늘리는 일이 아니라, 실제 사용자 의도를 더 잘 나누는 일이다. RFP 사용자는 “이거 뭐 하는 사업이야?”, “제안서에 뭘 강조해야 해?”, “마감일과 예산은?”, “이건 구축이야 운영이야?”, “최종 업체도 알 수 있어?”처럼 묻는다. 시스템은 그중 답할 수 있는 것과 멈춰야 하는 것을 구분해야 한다. 다음 루프는 이 지도를 바탕으로 v10 작은 묶음을 고정하고, 답변 보기 전에 metadata와 본문 노출 여부를 먼저 확인하는 쪽이 맞다.

진행 전에 내려받은 `rfp_question_bank.md`와 `rfp_question_bank.csv`를 열어 보았다. 총 95문항이었다. 처음에는 L116에서 말했던 12~16개 정도의 작은 묶음이 떠올랐지만, 사용자의 지적이 맞았다. 지금 필요한 것은 숫자를 예쁘게 맞추는 일이 아니라, 기존에 이미 여러 번 본 패턴을 걷어내고 아직 보지 못한 질문 신호를 남기는 일이다.

그래서 이번에는 95개를 한 번에 평가하지 않고, 먼저 겹치는지부터 나눴다. 단일문서 요약, 익숙한 기관명 별칭, 기존 다중문서 비교, 이미 많이 다룬 낙찰업체나 평가위원 같은 문서 밖 정보는 일단 잘랐다. 반대로 전체 코퍼스 집계, 예산 순위, 발주기관 반복, 파일 형식 집계, 계약/입찰 조건, API/인프라/UI/QA/DevOps 같은 기술검토, 임원용 요약과 고객 메시지 같은 사용자수준 변환, top-k 출처와 latency 같은 RAG 운영 질문은 남겼다.

결과는 95개 중 68개 유지, 27개 보류였다. 다만 이 68개도 하나의 점수로 섞으면 안 된다. 어떤 질문은 일반 RAG 답변이고, 어떤 질문은 CSV나 metadata 도구가 있어야 답할 수 있고, 어떤 질문은 직전 대화의 선택 사업이 있어야 의미가 생긴다. 또 top-k 청크나 filter on/off 비교, 응답 시간 분석은 답변 생성 문제가 아니라 실험 장치 문제다. 이걸 한 EDD로 섞으면 점수가 오른 것처럼 보여도 실제로는 서로 다른 시험지를 한 장에 붙인 셈이 된다.

그래서 새 결론은 이렇다. 12~16은 실행 단위의 권장 크기일 수는 있지만, 발견 단계의 제한선은 아니다. 안 해본 신호는 남겨야 한다. 대신 lane을 나누고, lane마다 질문 파일을 고정한 뒤, 원인과 결과와 다음 근거를 따로 기록해야 한다. 오늘 만든 산출물은 `eval/question_bank_gap_review_20260707.md`, `.json`, `.csv`다. 다음 작업은 유지된 후보를 lane별 실행 파일로 나누고, “사용자가 지정한 사업”으로 되어 있는 질문에는 먼저 구체 사업 seed를 붙이는 것이다.

68개를 한 번에 돌리면 터질 가능성이 높다는 판단은 맞았다. 문항 수만 많은 문제가 아니라, 서로 다른 시험이 한 바구니에 들어 있었다. 그래서 먼저 4개를 뺐다. Q092는 metadata filter on/off 비교라서 실험 장치 비교에 가깝고, Q093은 latency 측정이라 답변 문제가 아니라 계측 문제였다. Q094는 틀린 답변이 나온 뒤에야 의미가 있는 실패 원인 분석이고, Q095는 질문세트 분류라 지금 우리가 하고 있는 설계 작업과 겹쳤다. 그래서 이 네 개는 버린 것이 아니라, RAG 운영 하네스 쪽으로 보류했다.

남은 64개는 16개씩 네 묶음으로 나눴다. A는 전체탐색과 코퍼스 분석, B는 계약/입찰과 기술추출, C는 기술검토 후반부와 후속질문 및 경계질문, D는 사용자수준 변환과 사업판단 및 근거표시 쪽이다. 그냥 내가 나누고 끝내지 않고, 네 역할 검토자에게 각각 하나의 묶음만 보게 했다. 결과적으로 4개 산출물 모두 정상 JSON으로 들어왔고, smoke도 통과했다.

검토에서 제일 좋은 지적은 A 묶음이었다. Q033~Q035 는 물리적으로 A에 들어 있지만, 사실 metadata 집계 아니라 다중문서 비교와 군집 요약에 가깝다. 그래서 옮길까 생각했는데, D 쪽과 바꾸면 그쪽의 사업판단 묶음이 더 흐려진다. 최종 결정은 물리적 16개 묶음은 유지하되, A 안에서 Q033~Q035 는 별도 subtotal로 빼서 읽는 것이다. 이게 더 정직하다. 숫자를 맞추려고 의미를 속이지 않는 쪽이다.

B와 C와 D도 조건이 붙었다. B의 Q038~Q050 은 "이 사업"이나 "해당 RFP"라고 묻기 때문에 실행 전에 반드시 구체 사업 seed가 있어야 한다. C의 후속질문도 직전 답변이나 선택 사업이 없으면 평가가 비어 버린다. D의 Q090~Q091 은 top-k 출처와 문서 근거 표시를 묻기 때문에, 검색 trace나 citation trace가 없으면 일반 답변 문항으로 돌리면 안 된다.

오늘의 결론은 "64개 준비 완료, 아직 답변 실행 아님"이다. 산출물은

`eval/question_bank64_shards_20260707.md`, `.json`, `.csv`, `.manifest.json` 이고, 검토 장부는

`eval/parallel_runs/20260707_144929_L118-question-bank-64-shard-preparation-and-team-review` 에 남겼다. 다음에 진짜로 돌리려면 먼저 shard별 runnable manifest를 만들어야 한다. seed 사업, seed turn, 본문에 보이는 필드와 metadata에만 있는 필드, trace가 필요한 문항을 모두 적고 나서야 첫 실행 점수를 말할 수 있다.

2026-07-07 오후 L119 - 64개를 바로 돌리지 않고, 무엇을 어떻게 돌릴지 먼저 고정했다

L118에서 64개 묶음은 만들어졌지만, 그 상태로 바로 실행하면 또 숫자가 사람을 속일 가능성이 컸다. 어떤 문항은 일반 RAG 답변이고, 어떤 문항은 CSV metadata를 집계해야 하고, 어떤 문항은 "이 사업"이라는 직전 선택이 있어야 하고, 또 어떤 문항은 top-k trace가 실제로 export되어야만 채점할 수 있었다. 그래서 오늘은 답변을 만들기 전에 runnable manifest를 먼저 만들었다. 점수를 올리는 작업처럼 보이지 않을 수 있지만, 사실은 다음 점수가 거짓말을 하지 않게 만드는 작업이다.

원본 100건을 다시 읽어서 seed 후보를 잡았다. 고려대학교 차세대 포털 사업은 큰 교육/포털 사업이라 계약 조건과 사용자수준 변환에 좋고, 국방과학연구소 대용량 자료전송은 기술 요구사항 확인에 좋았다. 한국가스공사 ERP는 큰 통합정보시스템이라 예산과 연계, 이관 쪽에 쓸 만했고, 봉화군 재난통합관리시스템은 보안과 운영 관점에서 쓸 수 있었다. 중이온가속기 극저온 운전, 의료기기 종합정보시스템 두 건, 한국원자력연구원 선량평가시스템, 금액이 비어 있는 재생에너지 지원센터까지 seed catalog에 넣었다.

그다음 64문항을 실행 방식별로 다시 표시했다. 일반 EDD 후보는 49개, metadata나 trace처럼 별도 길로 가야 하는 문항은 15개였다. Q001~Q012 와 Q065 는 일반 답변 점수에 섞으면 안 되고, 코퍼스 집계용 sidecar로 봐야 한다. Q090~Q091 은 더 엄격하다. 이 문항은 "어떤 chunk가 검색됐는가", "어느 부분을 근거로 썼는가"를 묻기 때문에 trace wrapper가 없으면 실행 준비가 안 된 것이다. 그래서 이 둘은

`blocked_until_trace_wrapper` 로 내렸다. 이건 후퇴가 아니라 정직한 표시다.

네 역할 검토도 다시 붙였다. A는 keyword sidecar가 너무 넓게 잡힐 수 있다고 했다. 그래서 keyword hit를 정답 gold로 보지 않고 후보 preflight로만 쓰기로 했다. B는 Q038~Q045 에서 두 seed를 섞으면 안 된다고 했다. "선택한 사업"을 묻는 질문에 고려대와 한국가스공사 내용을 섞어 답하면 점수는 그럴듯해도 실제로는 틀린 답이다. 그래서 한 실행에는 seed 하나만 쓰고, 없는 필드는 없는 필드로 말하게 했다. C

는 그대로 통과했다. D는 Q090~Q091 trace wrapper와 사용자수준/사업판단 문항의 별도 게이트를 강하게 요구했다.

작업 중에 작은 사고도 있었다. 처음 worker 산출물이 내용상으로는 쓸 만했지만, 표준 JSON 계약 형태가 아니어서 smoke가 실패했다. 이걸 보고 원본 리뷰는 worker_outputs/raw_reviews에 보존하고, 상위 산출물은 parallel_team_worker_output.v1 형태로 감싸 다시 검증했다. 두 번째 smoke는 통과했고, 이 실패 자체도 좋은 교훈이었다. 평가 루프에서는 모델 답변만 테스트하면 안 된다. 검토자가 남긴 산출물이 기계적으로 검증 가능한지도 품질의 일부다.

오늘 산출물은 eval/question_bank64_runnable_manifest_20260707.json, .csv, .md, .manifest.json 이다. 결론은 "62개는 준비됨, 2개는 trace wrapper 전까지 막힘, 그래도 전체 64개를 하나의 EDD로 읽으면 안 됨"이다. 다음 단계는 metadata sidecar runner와 selected-project runner를 만들고, blocked trace 문항은 wrapper가 생기기 전까지 별도 표식으로 남기는 것이다. 여기까지 해두면 다음 실행에서 점수가 오르거나 떨어졌을 때, 그것이 모델 성능인지 측정 방식인지 훨씬 덜 헷갈리게 된다.

2026-07-07 오후 L120 - 돈 안 쓰는 집계 문항부터 실제로 돌릴 수 있게 만들었다

L119에서 "metadata sidecar가 필요하다"고 말만 해두면 아직 반쪽이었다. 그래서 이번에는 API를 부르지 않고, 원본 data_list.csv와 L119 manifest만으로 집계 문항을 실행하는 runner를 만들었다. 이름은 scripts/run_metadata_sidecar.py 다. 이걸 모델 답변을 잘하게 만드는 코드가 아니라, 모델이 답해야 할 때 옆에서 비교할 수 있는 집계 기준과 후보 목록을 만드는 코드다.

처음 실행은 완벽하지 않았다. Q070에서 하나가 빠졌다. 이유를 보니 Q070은 "그와 비슷한 교육 또는 학습 관련 사업"을 묻는 후속질문이라서, 자기 이름의 새 sidecar key가 있는 게 아니라 Q006의 교육/학습 후보군을 재사용하고 있었다. 그런데 첫 runner는 case id와 key id가 항상 같다고 가정했다. 그 가정이 틀렸다. 이걸 작은 버그지만 꽤 좋은 신호였다. 평가 도구도 질문의 참조 구조를 제대로 읽어야 한다는 걸 바로 드러냈기 때문이다.

그래서 runner를 고쳐 preflight_answer_key_ref를 직접 따라가게 했다. 다시 돌리니 17개가 모두 ready가 됐다. sidecar readiness score는 1.000이다. 다만 이 숫자는 "RAG가 잘 답한다"는 뜻이 아니다. "옆에서 비교할 집계 산출물이 준비됐다"는 뜻이다. 정확하게 말하면, 반복 발주기관, 상위 금액, 금액 기준, 파일 형식, 금액 누락 경계처럼 정확히 셀 수 있는 6개는 exact sidecar로 볼 수 있고, 교육/교통/재난/보건/문화/ODA/ERP/AI/보안 같은 keyword discovery류 11개는 후보 목록이다. 후보 목록은 row-level evidence를 다시 봐야 의미 정확도를 말할 수 있다.

검토자 둘을 붙여서 runner와 보고서를 봤다. 하나는 strictness 라벨이 정직하다고 봤고, 하나는 Q070 -> Q006가 보고서에 더 분명히 보여야 한다고 했다. 맞는 지적이라 결과 JSON과 CSV와 MD에 sidecar_key_id를 넣고, MD에는 "Intentional Sidecar Aliases" 섹션을 추가했다. 이제 Q070은 누가 봐도 실수로 Q006이 들어간 게 아니라, follow-up domain expansion이 Q006 후보군을 의도적으로 재사용한다는 것을 알 수 있다.

오늘의 결론은 “metadata 쪽은 실행 준비가 됐다”이다. 하지만 이것도 일반 EDD와 섞으면 안 된다. L120의 1.000은 모델 점수가 아니라 sidecar 준비 점수다. 다음으로 가장 좋은 작업은 selected-project runner다. Q038~Q053, Q067, Q069~Q073, Q077~Q089처럼 “이 사업”이나 직전 답변을 전제로 하는 문항은 seed 하나를 고정하고, 직전 turn을 만들어주고, 서로 다른 사업의 근거를 섞지 못하게 해야 한다. 이걸 해두면 그다음에는 진짜 답변 실행을 더 덜 위험하게 돌릴 수 있다.

2026-07-07 오후 L121 - “이 사업”이 무엇인지 먼저 못 박았다

metadata 쪽이 준비되자 다음으로 남은 위험은 “이 사업”과 “방금 말한 것”이었다. 이런 질문은 그냥 한 문장으로 던지면 평가가 이상해진다. 사용자는 이미 앞에서 어떤 공고를 골랐다고 생각하는데, 실행기는 그 맥락을 모를 수 있다. 그러면 검색기가 비슷한 공고를 섞어 가져오고, 모델은 그럴듯하게 답한다. 점수는 나와도 실제 대화 성능은 아닐 수 있다.

그래서 L121에서는 selected-project batch builder를 만들었다. 답변을 생성한 게 아니라, 답변을 생성하기 전에 대화의 바닥을 깔아둔 것이다. 계약/입찰 문항 Q038~Q045는 고려대학교 차세대 포털 사업으로 고정했다. 기술 문항 Q046~Q053은 한국가스공사 ERP와 봉화군 재난통합관리시스템을 나눠 붙였다. 후속질문과 사용자수준 문항은 고려대 포털, 스포츠윤리센터 LMS, 그리고 ISP/ISMP 문맥이 필요한 Q073에는 새로 인천일자리플랫폼 ISP seed를 붙였다.

중요한 산출물은 두 종류다. 하나는 questions_l121_selected_project_multiturn_primary.json이다. 이건 진짜 대화처럼 먼저 “이 공고를 선택한 사업으로 보자”고 말하고, 그다음 질문을 던진다. 이걸 다 실행하면 71번 호출이 필요하다. 다른 하나는 questions_l121_selected_project_resolved_primary.json이다. 여기서는 seed를 질문 안에 풀어 넣어서 한 번에 묻는다. 이건 35번 호출이면 된다. 싸게 smoke를 볼 때는 resolved가 좋지만, 후속질문 기억력이나 대화 맥락을 말하려면 multiturn을 써야 한다. 둘은 같은 점수가 아니다.

검토 과정에서도 좋은 지적이 나왔다. 계약 seed는 고려대 하나로 고정되어 있어서 좁지만, 이번 목적이 “선택한 사업을 섞지 않기”라면 괜찮다. 기술 seed는 ADD 대용량 자료전송 secondary variant를 만들었는데, 이것은 본증거가 아니라 sparse-field 진단용이다. 만약 이걸 본 점수에 섞으면 다시 오염된다. 후속/persona 쪽은 Q067~Q072와 Q073 구조가 괜찮지만, resolved one-turn은 메모리 성능을 충분히 못 본다는 경고가 붙었다.

작은 운영 이슈도 기록했다. 기술 seed 검토자는 처음에 모델 capacity 문제로 실패했고, 다른 worker로 다시 시켜 완료했다. 또 계약 검토자가 JSON 전체 파싱 실패를 말했는데, 확인해보니 PowerShell에서 UTF-8 옵션 없이 읽어서 깨진 오탐이었다. Python 전체 파싱과 `Get-Content -Encoding UTF8 | ConvertFrom-Json`은 모두 통과했다. 평가 루프에서는 이런 도구 검증 실패도 그냥 넘기면 안 된다. 원인이 데이터인지, 인코딩인지, 진짜 JSON 오류인지 구분해야 한다.

오늘 결론은 “selected-project 실행 준비 완료, 아직 답변 실행 아님”이다. primary 35개, secondary 16개가 준비됐고, cheap smoke는 35콜, 진짜 대화 평가는 71콜이다. 다음은 바로 전체를 돌리는 것보다 작은 resolved smoke를 먼저 하거나, secondary technical variant를 돌리기 전에 sparse-field not-found guard를 만드는 쪽이 맞다. 이 순서가 돈도 아끼고, 점수가 왜 올랐는지도 덜 헛갈리게 한다.

2026-07-07 오후 L122 - 새 문항은 모델보다 먼저 평가 장치를 시험했다

새로 들어온 문항들을 보면서 내가 놓치면 안 되는 점이 있었다. 이 질문들이 바로 답변 품질만 때리는 게 아니라, 우리가 무엇을 점수로 볼 수 있는지부터 흔들었다는 점이다. Q090~Q091은 답변만으로는 안 된다. 검색 trace와 chunk 출처 지도가 있어야 한다. Q001~Q012, Q065, Q070 같은 문항은 일반 답변 점수라기보다 corpus 집계나 후보 sidecar 쪽이다. Q070은 자기 key가 아니라 Q006 후보군을 의도적으로 다시 쓰는 구조라서, case id와 정답 key가 항상 같다는 가정도 깨졌다.

그래서 이번에는 바로 답변을 돌리지 않고, L121에서 남긴 sparse-field guard를 먼저 만들었다. 대상은 ADD 대용량 자료전송시스템을 seed로 쓰는 기술 검토 변형 Q046~Q053이다. 처음에는 CSV의 텍스트 컬럼만 보고 필드가 있는지 살폈다. 결과는 꽤 sparse하게 나왔다. 31개 필드 그룹 중 4개만 보이고 27개는 없는 것으로 잡혔다. 이대로라면 “없는 API, DB, 클라우드, 장애복구를 지어내지 말라”는 좋은 시험처럼 보였다.

그런데 raw HWP를 직접 파싱해서 다시 보자 결과가 완전히 뒤집혔다. 31개 필드 그룹이 모두 원문에서 보였다. API, 데이터 이관, 보안장비, 로그, 권한, 암호화, 모니터링, 장애복구 같은 단어들이 실제 원문 안에 있었다. 즉 ADD 문항은 raw 원문 기준으로는 빈 필드 시험이 아니라, CSV가 짧아서 비어 보였던 것이다. 이건 작은 발견이 아니라 꽤 큰 발견이다. “문서에 없음”이라는 판정이 모델 실패가 아니라 근거 원천 선택의 실패일 수 있기 때문이다.

검토 역할에서도 중요한 지적이 나왔다. secondary variant 16개가 아직 ordinary_edd_candidate=true와 EDD route를 달고 있었다. 이걸 그대로 실행하면 진단용 변형이 본점수에 섞일 수 있다. 그래서 L122는 EDD 개선 지점이 아니라 오염 방지 지점으로 남긴다. 산출물은 scripts/run_sparse_field_guard.py와 L122 폴더의 analysis/sparse_field_guard 아래에 있다. 스모크도 통과했다. 다음에는 secondary row가 scoreboard에 절대 들어가지 못하게 aggregator나 registry 쪽에 차단 장치를 넣거나, raw 원문 기준으로 정말 sparse한 기술 seed를 먼저 찾아야 한다.

2026-07-07 오후 L123 - 점수표 입구에 문지기를 세웠다

L122에서 제일 찜찜했던 건 secondary variant가 걸보기에는 진단용인데, 파일 안에는 아직 본점수 후보처럼 보이는 값이 남아 있다는 점이었다. 그래서 이번에는 답변을 돌리지 않고 점수표 입구를 먼저 막았다. scripts/aggregate_parallel_eval.py에 secondary variant 감지 규칙을 넣었다. 파일명에 secondary_variants나 secondary_technical_guarded가 있거나, 질문 파일 안에 variant_claim, secondary 성격의 diagnostic_label, secondary 관련 promotion blocker가 있으면 무조건 diagnostic_only로 내려가게 했다.

검증은 일부러 얄밟게 했다. edd_score=99.99, coverage/MRR/judge가 모두 완벽한 가짜 행을 만들고, 그 행이 L121 secondary 질문 파일을 가리키게 했다. 만약 이 행이 scoreboard에 들어가면 앞으로도 높은 점수를 보고 착각할 수 있다. 결과는 의도대로였다. rows는 1개였지만 scoreboard는 0개, diagnostic-only는 1개였다. 상태도 diagnostic_secondary_variant로 찍혔다.

이건 성능이 오른 게 아니다. 하지만 보고서 신뢰도는 오른다. 점수는 마지막 집계에서 말이 되거나 말이 안 되거나가 결정된다. 그래서 모델을 고치기 전에 점수표의 문을 고친 셈이다. 다음에는 raw 원문 기준

으로 정말 빈 필드가 많은 기술 seed를 찾거나, 작은 답변 smoke를 돌리더라도 secondary row는 promotion에서 빠진다는 전제를 지켜야 한다.

2026-07-07 오후 L124 - 진짜 빈 필드가 있는 문서를 다시 찾았다

ADD가 sparse할 거라는 생각은 raw 원문 앞에서 무너졌다. 그래서 이번에는 아예 전체 기술 후보를 다시 훑었다. 목표는 "질문이 어려운가"가 아니라 "문서 자체가 어떤 필드는 답할 수 있고 어떤 필드는 답할 수 없는가"를 먼저 찾는 것이었다. 그래야 모델이 빈칸을 채우는지, 아니면 정직하게 없다고 말하는지 볼 수 있다.

처음 기준은 또 조금 틀렸다. 보이는 필드가 10개 이하인 문서만 후보로 잡으려 했더니 후보가 0개가 됐다. 생각해보니 너무 단순했다. sparse 진단에 필요한 건 "거의 아무것도 없는 문서"가 아니라, 답할 수 있는 필드와 답할 수 없는 필드가 함께 있는 문서다. 그래서 기준을 "없는 필드가 충분히 많은가"로 바꾸었다. 그 결과 raw 원문 기준 97개 기술 후보 중 5개가 잡혔다.

가장 좋은 후보는 D0C073 우즈벡-키르기즈스탄 기후변화대응 스마트 관개시스템 구축사업이다. 31개 기술 필드 중 15개는 보이고 16개는 비어 있다. 두 번째는 D0C051 중이온가속기용 극저온시스템 운전 용역이고, 세 번째는 D0C025 용인 첨단 시스템반도체 국가산단 용수공급사업 타당성조사 및 기본계획 수립 용역이다. 이 셋은 다음 sparse 진단 질문 파일을 만들기에 좋다.

이번에도 답변 실행은 하지 않았다. 대신 seed를 잘못 고르면 평가가 시작부터 흔들린다는 걸 확인했다. 다음에 할 일은 이 상위 후보 2~3개로 작은 frozen 질문 파일을 만들고, 첫 답변은 raw로 보존하는 것이다. 그 결과가 좋은 나쁜진단용으로 남겨야 한다.

2026-07-07 16:00 L125 - 정밀 점검용 질문 세트 고정

- 원인 기록: L124에서 raw 텍스트 기준으로 선택한 D0C073, D0C051, D0C025는 보이는 항목과 없는 항목이 뒤섞여 있어, 단순 노출 부족이 아닌 경계 위반 유무를 보는 데 맞다.
- 조치: questions_l125_sparse_field_diagnostic_frozen 6건을 만들고 mixed + padding_trap으로 구분해 froze 상태로 남겼다.
- 상태: 이번 단계는 실행이 아니라 질문-증거 자료 고정 단계다. 점수는 갱신되지 않았고, 스코어보드 반영 대상도 아니다.
- 다음 루프: L125 6건에 대해 no-judge 진단 실행을 먼저 돌려 경계 누수(padding_trap)가 몇 건인지, 어떤 케이스에서 지원 증거가 없는데도 빈칸을 채우는지 기록한 뒤에만 하드 가드 규칙을 추가한다.
- 반성 포인트: 새 질문을 만들 때마다 성과표가 오르는지보다, 이번처럼 경계 누수 원인과 처리 방식이 남는지가 더 중요했다.
- 기록 규칙 재확인: 원문 기준이 CSV냐 raw냐가 질문 신뢰도에 직접 영향을 주므로, 다음 루프에서도 소스 기준을 먼저 고정해 둔 뒤 같은 문항을 다시 채점한다.

2026-07-07 17:15 L126/L127 - sparse-field 진단 실행과 점수표 오염 방지

- 원인: L125는 질문 파일을 고정한 단계였고, 실제 답변을 봐야 `padding_trap` 에서 빈 필드를 채우는 지, 아니면 답할 수 있는 필드까지 전부 거절하는지 알 수 있었다.
- 병렬 실행: L126은 `baseline_default, prompt_sweep`, L127은 `topk_sweep, mmr_lambda_sweep` 으로 나눠 같은 6 문항을 돌렸다. 실행 중 폴더명이 `${stamp}` 로 생성되는 운영 실수가 있었고, 이후 `20260707_161721_L126...`, `20260707_161718_L127...` 로 정리했다.
- 처리: `scripts/aggregate_parallel_eval.py` 를 고쳐 `worker contract` 를 `utf-8-sig` 로 읽고, 질문 파일 내부의 `diagnostic_only`, `claim_use`, `promotion_blocker` 를 보고 `scoreboard` 승격을 막게 했다. 이 수정 후 L126/L127은 모두 `scoreboard_rows=0`, `diagnostic-only`로 재집계됐다.
- 결과: L126에서 보기상 가장 좋은 후보는 `prompt_concise_verified` 였다. EDD는 76.64, `false-abstain`은 0.0, 평균 지연은 28.648초였다. 다만 이것은 진단 점수이지 대표 성능 점수가 아니다.
- 결과: L127에서는 `lambda05_top8_filter_rewrite_control` 이 EDD 71.12로 제일 높았지만, `false-abstain` 0.5라 `prompt` 쪽보다 약했다. 검색 파라미터를 바꾸는 것만으로 `sparse-field` 문제가 풀리지는 않았다.
- 추가 감리: `scripts/audit_sparse_answer_runs.py` 를 만들어 일반 EDD와 별도로 `sparse` 답변 모양을 봤다. `prompt_concise_verified` 는 `padding risk 0`, `detector conflict 0`이었지만 `pass rate`는 0.5, `present-field coverage`는 0.542였다.
- 인사이트: 이제 문제는 "없는 필드를 지어내는가" 하나가 아니다. 좋은 답은 보이는 필드는 답하고, 없는 필드는 없다고 말해야 한다. 일반 `relevance judge`는 사용자가 억지로 빈칸을 채우라고 한 질문에서 안전하게 거절한 답을 낮게 볼 수 있고, `binary abstention`도 부분 답변을 전역 거절로 오해할 수 있다.
- 다음: L125 `sparse` 문항에 `field-level expectation`을 붙여 `present-field` 회수와 `not-found` 정확도를 직접 재야 한다. 그 다음에야 `prompt_concise_verified` 를 후보로 좁게 재실행할 수 있다. 지금 `broad top_k sweep`을 더 돌리는 것은 개선보다 노이즈에 가깝다.